

SPRINT 3

Patricio – tu amigo robot

Presentado por:

Mari Dapcheva
Adenor Buret
Pablo Meana
Cesar Herrero
Santiago Aguirre
Juan Bautista Peris



Índice:

1.Historial de versiones	7
2.Introducción	9
2.1.Descripción del proyecto	9
2.1.2.Estado del arte	9
2.1.3.Bee-Bot (TTS Group)	9
2.1.4.Miko 3 (Emotix)	10
2.1.5.Moxie (Embodied)	10
2.2.Propuesta de valor de Patricio	11
2.3.Objetivos del proyecto	11
2.3.1.Pilar 1 — Navegación autónoma (Nav2)	12
2.3.2.Pilar 2 — Interfaz web de control	12
2.3.3.Pilar 3 — Uso de OpenCV	13
2.3.4.Pilar 4 — Reconocimiento (IA)	14
2.3.5.Resumen consolidado	15
3.Paquetes	16
3.1.Diseño de paquetes del sistema	16
3.2.Objetivos del diseño	16
3.3.Estructura modular del sistema	16
3.4.Comunicación entre paquetes	19
3.5. Justificación global del diseño	20
3.5.1. patricio_my_world	20
3.5.1.1. Estructura del paquete	20
3.5.1.2. Descripción	21
3.5.1.3. Topics	22
3.5.1.4 Nodos	22
3.5.2.patricio_nav_punto	23
3.5.2.1. Estructura del paquete	23
3.5.2.2. Descripción	24
3.5.2.3. Topics	24
3.5.2.4. Nodos	25
3.5.3.patricio_captacion	25
Estructura del paquete	25
Descripción	26



3.5.4.patricio_interfaces_____	26
3.5.4.1.Estructura del paquete_____	26
3.5.4.2.Descripción_____	27
3.5.4.3.Servicios_____	27
3.5.4.3.1. Request_____	27
3.5.4.3.2. Response_____	27
3.5.5.patricio_pilla_pilla_____	27
3.5.5.1. Estructura del paquete_____	27
3.5.5.2.Descripción_____	28
3.5.5.3. Topics_____	28
3.5.5.4.Nodos_____	29
3.5.6.patricio_escondite_____	30
3.5.6.1. Estructura del paquete_____	30
3.5.6.2. Descripción_____	31
3.5.6.3. Topics_____	32
3.5.6.4. Nodos_____	33
3.5.6.5. Servicios_____	34
3.5.7.patricio_web_____	35
3.5.7.1. Estructura del paquete_____	35
3.5.7.2. Descripción_____	36
3.5.7.3. Topics (usados desde la web)_____	36
3.5.7.4. Servicios_____	36
4.Mapa y navegación_____	37
4.1.Mapa_____	37
4.2.Estructura del Mapa_____	37
4.2.1. Uso por Patricio_____	38
5. Diseño Backend_____	38
5.1. Base de Datos_____	38
5.2. Lógica de visión artificial_____	40
6. Stack Tecnológico_____	40
7. Nodos_____	40
8. Arquitectura del sistema_____	42
8.1. Diagrama de la Arquitectura del sistema_____	42
8.1.1.Capa 1 — Usuario_____	42
8.1.1.1.Flecha — WebSocket, puerto 9090_____	43
8.1.2.Capa 2 — Comunicación_____	43



8.1.2.1.Flecha — Mensajes ROS 2	44
8.1.3.Capa 3 — Robot	44
9.Interfaz Web del sistema	47
9.1.Identidad visual (Moodboard)	47
9.2.Referencias visuales	48
9.3.Paleta de colores	48
9.4.Tipografía	48
9.5.Pantallas	49
9.5.1.Landing page - Inicio	49
9.5.1.1.Características:	50
9.5.2.Panel de control (Dashboard)	50
9.5.3. Panel de juegos	51
9.5.3.1. Configuración de conexión	51
9.5.3.2. Controles de movimiento	51
9.5.3.2.1.Movimiento básico:	51
9.5.3.2.2.Control manual (WSAD):	51
9.5.4.Monitor del robot	52
9.5.5. Navegación	52
9.5.6. Visualización	52
9.5.7.Sección de presentación (Hero Section)	53
9.5.8. Formulario de registro	54
9.5.8.1.Campos del formulario	55
9.5.8.2.Validaciones implementadas	55
9.5.8.2.1. Funcionalidad	55
9.5.8.2.2. Características de diseño	55
9.5.9. Formulario de inicio de sesión	56
9.5.9.1. Campos del formulario	56
9.5.9.2. Validaciones implementadas	57
9.5.9.3. Funcionalidad	57
9.5.9.4. Características de diseño	57
9.5.10. Política de privacidad	57
9.5.10.1. Contenido principal	57
9.5.10.2. Derechos del usuario	58
9.5.10.3. Otros aspectos	58
9.5.10.4. Funcionalidad	58



9.5.11. Contactanos_____	58
9.5.11.1. Información mostrada_____	58
9.5.11.2. Funcionalidad_____	59
9.5.11.3. Integración en la web_____	59
9.6. API interna y comunicación_____	59
9.6.1. Mapa de Tópicos, Servicios y Mensajes_____	59
9.6.2. Pantalla de caras del robot_____	59
9.6.2.1. Expresiones diseñadas_____	60
9.7. Indicador de actividad_____	62
9.8. Mecanismo de activación_____	62
10. Tópicos_____	63
10.1.Tópicos — Confirmados en ejecución_____	63
10.2. Tópicos — Pendientes (módulos de juego/navegación)_____	64
11. Nodos activos confirmados_____	66
12. Servicios (Rosbridge / Rosapi)_____	66
13. Ejemplos de JSON desde JavaScript_____	67
14. Lista de Dependencias del Sistema_____	70
14.1.Sistema Operativo y ROS 2_____	70
14.2.Paquetes ROS 2 Core_____	71
14.3. Rosbridge — Confirmado activo_____	71
14.4. Simulador Gazebo — Confirmado activo_____	71
14.5. Navegación Nav2_____	72
14.6. Librerías WEB - Frontend_____	72
15. Instalación completa desde cero_____	72
15.1. Activar ROS 2_____	72
15.2. Clonar y compilar_____	73
15.3. Lanzar simulación (Terminal 1)_____	73
15.4. Lanzar Rosbridge (Terminal 2)_____	73
15.5. Acceder a la interfaz web_____	73
16. Verificación del sistema (pruebas y validación)_____	73
16.1. Lista de pruebas_____	73
16.1.1. H11-P1 Conexión a robot desde la web_____	75
16.1.2. H7-P1 - Iniciar el juego Pilla-pilla_____	76
16.1.3. H7-P2 - Detener el juego Pilla-pilla_____	78
16.1.4. H11-P1 - Iniciar el juego Escondite_____	79



16.1.5. H11-P2 - Detener el juego Escondite_____	81
16.1.6. H11-P2 - Login correcto_____	82
16.1.7. H11-P3 - Login incorrecto_____	83
16.1.8. H11-P4 - Cerrar sesión_____	84
16.1.9. H11-P5 - Comunicación entre la web y la API_____	85
16.1.10. H14-P1 - Navegar a un punto_____	86
17. Troubleshooting_____	88
17.1. La web muestra "Disconnected"_____	88
17.2. No conecta la web con el robot_____	88
17.3. El robot no se mueve_____	88
17.4. El juego Pilla-Pilla no inicia_____	89
17.5. El robot no llega a los waypoints_____	89
17.6. No se ve el mapa en la web_____	89
17.7. El login no funciona_____	89
18. Conclusión_____	89



1. Historial de versiones

Versión	Autor	Descripción	Fecha
v1.0	Pablo Meana	Versión inicial del documento. Creación de la estructura general, redacción de la concepción del producto y descripción de los paquetes ROS 2 del sistema.	30/03/2026
v1.1	Mari Dapcheva	Ampliación del documento con la descripción de la arquitectura del sistema, la interfaz web, la base de datos, la API interna, el plan de verificación de funcionalidades y el apartado de troubleshooting.	20/04/2026
v1.2	Juan Bta. Peris	Incorporación de la base teórica del proyecto: introducción con el estado del arte de robots educativos referentes, contexto del proyecto y definición de los	7/05/2026



		objetivos SMART de los cuatro pilares (Navegación Nav2, Interfaz Web, OpenCV y Reconocimiento IA).	
--	--	---	--



2.Introducción

2.1.Descripción del proyecto

Durante la fase de concepción del producto, realizamos un análisis exhaustivo del mercado y las necesidades de los usuarios con el objetivo de desarrollar una solución innovadora en el ámbito de la educación infantil.

A través de vigilancia tecnológica, ideación colaborativa y validación de usuario, identificamos una oportunidad clara: la falta de atracción al aprendizaje.

Así nació la idea de Patricio, como un robot educativo orientado a niños de 4 a 6 años, con el objetivo de fomentar el aprendizaje mediante interacción lúdica e integración de inteligencia artificial para una experiencia autónoma y adaptativa.

La propuesta se apoya en un público objetivo claro (educación infantil temprana) con unas funcionalidades previstas descritas en el README del proyecto: matemáticas y alfabeto, narración de cuentos y chistes, juegos de movimiento ("pilla pilla", "el pollito inglés"), expresión en pantalla, navegación autónoma y reconocimiento de voz.

2.1.2.Estado del arte

Durante la fase inicial del proyecto se realizó un ejercicio de vigilancia tecnológica para identificar las soluciones existentes en el ámbito de la robótica educativa orientada a edades tempranas. El análisis se centró en aquellas plataformas que comparten parte del público objetivo o de las funcionalidades de Patricio, con el fin de detectar el hueco de mercado que justifica el proyecto y de establecer la propuesta de valor diferencial. Se han seleccionado tres referentes representativos del estado actual del sector, cada uno cubriendo un nicho distinto del problema.

2.1.3.Bee-Bot (TTS Group)

Bee-Bot es uno de los robots educativos más extendidos en las aulas de educación infantil y primer ciclo de primaria en España. Diseñado con forma de abeja y orientado a niños a partir de tres años, su finalidad es introducir los conceptos básicos de programación, pensamiento computacional y razonamiento secuencial. Su funcionamiento se basa en una botonera de cuatro



direcciones situada en el dorso del robot, mediante la cual el niño define una secuencia de hasta doscientos movimientos que el dispositivo ejecuta sobre una cuadrícula impresa en una alfombrilla temática.

Aunque Bee-Bot cumple con solvencia su misión didáctica en lo relativo a lateralidad, ubicación espacial y resolución de problemas, presenta limitaciones evidentes para un proyecto más ambicioso: carece de navegación autónoma, no incorpora inteligencia artificial ni capacidades de reconocimiento, no permite interacción verbal con el niño y no ofrece ninguna interfaz para el seguimiento por parte del tutor o el educador.

2.1.4.Miko 3 (Emotix)

Miko 3 representa el extremo opuesto del espectro: un robot sobremesa con pantalla expresiva orientado a niños de cinco a diez años, que apuesta de forma decidida por la inteligencia artificial conversacional. Incorpora procesamiento de lenguaje natural, reconocimiento de voz y reconocimiento facial, una cámara gran angular y un motor de inteligencia emocional que identifica y recuerda los estados de ánimo del niño. Su catálogo de contenidos incluye juegos educativos, historias, programación visual y aplicaciones de marcas como Disney.

Sus principales limitaciones desde la perspectiva del proyecto son tres: se trata de una plataforma cerrada y propietaria que no permite extensión por parte del docente; el robot es estático y no se desplaza por el entorno del niño, por lo que la interacción es siempre sedentaria y centrada en una pantalla; y la franja de edad recomendada queda por encima del público objetivo de Patricio.

2.1.5.Moxie (Embodied)

Moxie, desarrollado por la empresa Embodied, es un robot social diseñado para apoyar el aprendizaje socioemocional de niños de cinco a diez años a través de misiones semanales sobre amistad, empatía o autorregulación emocional. Combina reconocimiento facial y de voz, una pantalla con expresiones faciales animadas y un currículo elaborado por especialistas en desarrollo infantil. Al igual que Miko, comparte tres limitaciones que abren un espacio para una propuesta alternativa: el ecosistema es propietario, el robot es estático y el rango de edad recomendado deja fuera la franja de cuatro a seis años en la que se concentran las necesidades educativas que motivan este proyecto.



2.2.Propuesta de valor de Patricio

El análisis comparado revela una oportunidad clara en el mercado: no existe actualmente una solución de robótica educativa orientada al rango de cuatro a seis años que combine, de forma simultánea, inteligencia artificial conversacional, reconocimiento por visión por computador, navegación autónoma por el entorno, juegos físicos que impliquen movimiento corporal del niño y una interfaz web abierta para el seguimiento por parte del tutor o el educador. Bee-Bot resuelve la programación elemental pero carece de IA y de navegación; Miko y Moxie resuelven la IA conversacional pero permanecen estáticos y se dirigen a una franja de edad superior. Patricio se sitúa precisamente en esa intersección no cubierta, apoyándose además en una pila tecnológica abierta basada en ROS 2 y Nav2 que garantiza la extensibilidad y la reutilización académica del trabajo realizado.

La siguiente tabla resume la comparativa entre los tres referentes y la propuesta de Patricio:

Característica	Bee-Bot	Miko 3	Moxie	Patricio
Rango edad	3-6 años	5-10 años	5-10 años	4-6 años
Navegación Autónoma	No	No (Estático)	No (Estático)	Si (Nav2)
Reconocimiento por IA	No	Voz y Facial	Voz y Facial	Voz y visión (OpenCV)
Juegos Físicos en el espacio	No	No	No	Sí (pilla-pilla, escondite,etc...)
Interfaz para tutor/educador	No	App Móvil	App Móvil	Web multirol
Plataforma	Firmware cerrado	SO Propietario	SocialX (propietario)	ROS 2 + Gazebo (abierto)

2.3.Objetivos del proyecto

Los objetivos del proyecto se articulan alrededor de los cuatro pilares funcionales definidos por la asignatura: navegación autónoma con Nav2,



interfaz web de control, uso de OpenCV y reconocimiento mediante inteligencia artificial. Para garantizar la trazabilidad y la verificabilidad de cada objetivo, su formulación sigue el criterio SMART, esto es, cada objetivo es específico, medible, alcanzable, relevante y temporalmente acotado.

2.3.1. Pilar 1 — Navegación autónoma (Nav2)

Objetivo: dotar a Patricio de la capacidad de desplazarse de forma autónoma por el entorno doméstico simulado, alcanzando puntos objetivo previamente definidos y evitando los obstáculos detectados por el sensor LiDAR, con una tasa de éxito igual o superior al noventa por ciento en el conjunto de pruebas de navegación del plan de validación, antes del final del Sprint 2 y verificado durante el Sprint 3.

- **Específico:** navegación punto a punto sobre un mapa cargado, utilizando AMCL para la localización y el stack Nav2 para la planificación y el control.
- **Medible:** porcentaje de pruebas superadas sobre el plan H14 (navegación a un punto) y H7 (navegación entre waypoints en el juego pilla-pilla).
- **Alcanzable:** la pila Nav2 sobre ROS 2 Jazzy está validada en el robot TurtleBot3, y el mapa del entorno ya está generado y depurado.
- **Relevante:** la navegación es la funcionalidad básica exigida por la asignatura y constituye la condición técnica que habilita los juegos físicos.
- **Temporal:** implementación durante el Sprint 2 y verificación documentada en el Sprint 3.

2.3.2. Pilar 2 — Interfaz web de control

Objetivo: desarrollar una interfaz web responsive que permita a un usuario sin conocimientos de ROS conectar con el robot, iniciar y detener los juegos disponibles, teleoperar el robot manualmente, enviar al robot a posiciones predefinidas, visualizar el mapa y la cámara en tiempo real, y gestionar su sesión, superando la totalidad de las diez pruebas funcionales definidas en el plan de validación durante el Sprint 3.

- **Específico:** frontend en HTML, CSS y JavaScript, backend Flask, comunicación con ROS 2 mediante rosbridge sobre WebSocket en el puerto 9090, y soporte de tres roles de usuario diferenciados (administrador, educador y familia).



- **Medible:** 10 de 10 pruebas funcionales del plan H7, H10, H11 y H14 superadas, con respuesta HTTP 200 verificable sobre la API.
- **Alcanzable:** las librerías roslibjs y ros2djs están consolidadas y disponibles como CDN, y la arquitectura propuesta desacopla el frontend del sistema ROS.
- **Relevante:** la interfaz es la condición indispensable para que el robot resulte utilizable por padres y educadores, que son los usuarios reales de la solución.
- **Temporal:** entrega y verificación dentro del Sprint 3.

2.3.3.Pilar 3 — Uso de OpenCV

Objetivo: incorporar al menos una funcionalidad de visión por computador implementada con OpenCV sobre la imagen publicada por la cámara del robot en el tópico `/camera/image`, integrada en el flujo de la interfaz web y verificable mediante una prueba documentada, antes del cierre del Sprint 4.

- **Específico:** pipeline OpenCV sobre Python en un nodo del paquete `patricio_captacion`, suscrito a `/camera/image` y publicador de un tópico de resultados de visión consumible desde la web.
- **Medible:** existencia del nodo, ejecución sin errores sobre un conjunto de imágenes de prueba y prueba funcional añadida al plan de validación.
- **Alcanzable:** la cámara del TurtleBot3 ya emite imágenes en el bridge configurado en el archivo `turtlebot3_burger_cam_bridge.yaml`.
- **Relevante:** la visión por computador es uno de los pilares exigidos por la asignatura y diferencia a Patricio de los referentes Bee-Bot, Miko y Moxie en el plano técnico.
- **Temporal:** implementación y verificación durante el Sprint 4.



2.3.4. Pilar 4 — Reconocimiento (IA)

Objetivo: integrar una capacidad de reconocimiento basada en inteligencia artificial que permita a Patricio identificar comandos de voz del niño y, opcionalmente, reconocer objetos o personas mediante el módulo de visión, alcanzando una tasa de aciertos igual o superior al ochenta por ciento sobre el dataset de validación construido por el equipo, antes del cierre del Sprint 4.

- **Específico:** módulo de reconocimiento de voz para comandos sencillos (iniciar juego, detener, repetir) y, según disponibilidad de tiempo, módulo de reconocimiento visual apoyado sobre el pipeline OpenCV del pilar anterior.
- **Medible:** tasa de acierto global $\geq 80\%$ sobre un conjunto de muestras de validación etiquetadas; la justificación de la métrica y la metodología de prueba quedan recogidas en el apartado de verificación de la memoria final.
- **Alcanzable:** existen bibliotecas y modelos preentrenados que cubren el reconocimiento de voz y la detección básica de objetos en tiempo real sobre la cámara del TurtleBot3.
- **Relevante:** constituye el núcleo de la propuesta de valor educativa y adaptativa del robot, y es precisamente la capacidad que Bee-Bot no incorpora y que Miko y Moxie sí ofrecen pero en una franja de edad superior.
- **Temporal:** implementación, integración con la navegación y la interfaz web, y verificación, durante el Sprint 4.



2.3.5. Resumen consolidado

La siguiente tabla recoge de forma sintética los cuatro objetivos del proyecto y permite verificar el cumplimiento del criterio SMART en cada uno de ellos.

Pilar	Específico (S)	Mediable (M)	Alcanzable (A)	Relevante (R)	Temporal (T)
Navegación Nav2	Desplazamiento autónomo en el entorno simulado evitando obstáculos.	Tasa de éxito $\geq 90\%$ en pruebas de navegación punto a punto.	Stack Nav2 sobre ROS 2 Jazzy ya validado en TurtleBot3.	Funcionalidad básica exigida por la asignatura y soporte del resto de pilares.	Sprint 2 (verificado en Sprint 3).
Interfaz Web	Control remoto y monitorización en tiempo real desde el navegador.	10/10 pruebas funcionales del plan de validación superadas.	Frontend HTML/JS + Flask + rosbridge (puerto 9090).	Permite operar el robot sin conocimientos de ROS.	Sprint 3 (entregable actual).
OpenCV	Procesamiento de imagen de la cámara para apoyo a la interacción.	≥ 1 pipeline OpenCV integrado y verificable desde la interfaz web.	Cámara del TurtleBot3 ya publicada en /camera/image .	Habilita la visión por computador que diferencia a Patricio de sus referentes.	Sprints 3-4.
Reconocimiento (IA)	Reconocimiento de voz y objetos/personas.	$\geq 80\%$ de aciertos en el dataset de validación construido por el equipo.	Combinación de modelos (voz y modelo de visión sobre OpenCV).	Núcleo de la propuesta de valor educativa y adaptativa.	Sprint 3-4 (integración con Nav2 y web).



3. Paquetes

3.1. Diseño de paquetes del sistema

El sistema del robot Patricio se ha diseñado siguiendo una **arquitectura modular basada en paquetes ROS 2**, con el objetivo de garantizar la escalabilidad, mantenibilidad y separación de responsabilidades.

Cada paquete cumple una función específica dentro del sistema, permitiendo desarrollar, probar y modificar componentes de forma independiente sin afectar al resto del sistema.

3.2. Objetivos del diseño

El diseño de paquetes se ha realizado con los siguientes objetivos:

- Separar claramente las responsabilidades del sistema
- Facilitar la reutilización de componentes
- Permitir el desarrollo paralelo entre distintos módulos
- Mejorar la mantenibilidad y depuración
- Asegurar una integración sencilla con ROS 2

3.3. Estructura modular del sistema

El sistema está dividido en los siguientes paquetes principales:

♦ **patricio_my_world**

Responsabilidad: Simulación del entorno del robot

Este paquete contiene todo lo necesario para ejecutar el entorno virtual en Gazebo, incluyendo:

- Modelos 3D del entorno (muebles, robot)
- Archivos URDF del robot
- Mundos de simulación (.world)



- Launch files para inicializar el sistema

Decisión de diseño:

Se ha separado la simulación en un paquete independiente para poder reutilizar el entorno en diferentes pruebas sin modificar la lógica del robot.

♦ **patricio_nav_punto**

Responsabilidad: Navegación autónoma

Este paquete implementa la navegación utilizando Nav2, permitiendo al robot:

- Localizarse en el mapa
- Planificar rutas
- Evitar obstáculos
- Desplazarse a puntos objetivo

Incluye:

- Mapas (.yaml y .pgm)
- Configuración de Nav2
- Launch files de navegación

Decisión de diseño:

Se ha aislado la navegación para poder usarla tanto en juegos como en otras funcionalidades sin duplicar lógica.

♦ **patricio_pilla_pilla**

Responsabilidad: Juego Pilla-Pilla

Implementa la lógica del juego mediante un nodo ROS 2 que:

- Genera objetivos de navegación
- Controla el estado del juego
- Se comunica mediante tópicos y servicios



Decisión de diseño:

Separar cada juego en su propio paquete permite añadir nuevos juegos en el futuro sin afectar al sistema existente.

♦ **patricio_escondite**

Responsabilidad: Juego Escondite

Este paquete implementa un sistema de búsqueda autónoma basado en posiciones dadas.

- Uso de servicios para iniciar/detener
- Separación entre lógica del juego y comunicación ROS

Decisión de diseño:

Se ha seguido una arquitectura modular separando la lógica del juego del interfaz ROS, mejorando la reutilización.

♦ **patricio_interfaces**

Responsabilidad: Definición de interfaces

Contiene los servicios personalizados del sistema, como:

- StartGame.srv
- IniciarEscondite.srv

Decisión de diseño:

Centralizar las interfaces evita duplicación y asegura coherencia en la comunicación entre paquetes.

♦ **patricio_captacion**

Responsabilidad: Captación de datos sensoriales

Preparado para gestionar:



- Sensores del robot
- Procesamiento de datos del entorno

Actualmente actúa como base para futuras ampliaciones.

Decisión de diseño:

Se ha creado como paquete independiente para facilitar la integración futura de sensores sin modificar otros módulos.

♦ **patricio_web**

Responsabilidad: Interfaz web y comunicación con el usuario

Incluye:

- Frontend (HTML, CSS, JS)
- Backend (API Flask)
- Comunicación con ROS mediante rosbridge

Permite:

- Control remoto del robot
- Inicio de juegos
- Monitorización en tiempo real

Decisión de diseño:

Separar la interfaz web del sistema ROS permite desacoplar completamente el frontend del backend robótico.

3.4.Comunicación entre paquetes

Los paquetes se comunican mediante los mecanismos estándar de ROS 2:

- **Topics:** para datos en tiempo real (posición, sensores)
- **Services:** para acciones concretas (iniciar juego, detener)
- **Actions:** para tareas largas (navegación)

Además, la web se conecta al sistema mediante:



- **WebSocket (rosbridge)**
 - Mensajes JSON → convertidos a ROS 2
-

3.5. Justificación global del diseño

El diseño modular adoptado permite:

- Escalar el sistema fácilmente (añadir nuevos juegos o sensores)
- Reutilizar componentes (navegación, interfaz)
- Aislar errores y facilitar la depuración
- Mejorar la organización del código

Este enfoque sigue las buenas prácticas de ROS 2 y arquitecturas distribuidas.

3.5.1. patricio_my_world

3.5.1.1. Estructura del paquete

```
patricio_my_world
├── CMakeLists.txt
├── include
│   ├── turtlebot3_drive.hpp
│   └── turtlebot3_drive.hppZone.Identifier
├── launch
│   ├── house.launch.py
│   ├── robot_state_publisher.launch.py
│   └── spawn_turtlebot3.launch.py
├── models
│   ├── Dining_Table
│   ├── Sofa
│   ├── Table_Marble
│   ├── turtlebot3_burger
│   ├── turtlebot3_burger_cam
│   └── tv_65in_emissive
```



```
|   └─ TVStand
|─ package.xml
|─ params
|   └─ turtlebot3_burger_bridge.yaml
|   └─ turtlebot3_burger_bridge.yamlZone.Identifier
|   └─ turtlebot3_burger_cam_bridge.yaml
|   └─ turtlebot3_burger_cam_bridge.yamlZone.Identifier
|─ rviz
|   └─ my_map_original.pgm
|   └─ my_map.pgm
|   └─ my_map.yaml
|─ src
|   └─ keyboard_teleop.cpp
|   └─ robot_pose_display.cpp
|   └─ turtlebot3_drive.cpp
|   └─ turtlebot3_drive.hpp
|─ urdf
|   └─ common_properties.urdf
|   └─ turtlebot3_burger_cam.urdf
|   └─ turtlebot3_burger.urdf
└─ worlds
    └─ house.sdf
    └─ house.world
```

3.5.1.2. Descripción

Este paquete se encarga de la **simulación del entorno del robot** utilizando Gazebo integrado con ROS 2.

En él se definen:

- El entorno (vivienda con mobiliario)
- El modelo del robot (URDF)
- Los objetos del escenario
- Los archivos de lanzamiento del sistema

Gracias a este paquete, es posible probar el comportamiento del robot en un entorno realista antes de su uso en un entorno físico.



3.5.1.3. Topics

Topic	Tipo de dato	Descripción
/cmd_vel	geometry_msgs/Twist	Control de velocidad del robot
/odom	nav_msgs/Odometry	Información de posición y velocidad del robot
/scan	sensor_msgs/LaserScan	Datos del sensor LiDAR para detección de obstáculos
/tf	tf2_msgs/TFMessage	Transformaciones dinámicas entre frames
/tf_static	tf2_msgs/TFMessage	Transformaciones estáticas del robot
/clock	roscpp_msgs/Clock	Tiempo de simulación de Gazebo

3.5.1.4 Nodos

Nodo	Paquete	Descripción
gz_sim	ros_gz_sim	Simulador que ejecuta el entorno `house.sdf`
robot_state_publisher	robot_state_publisher	Publica las transformaciones del robot a partir del URDF
spawn_entity	ros_gz_sim	Inserta el robot en el entorno de simulación
parameter_bridge	ros_gz_bridge	Conecta los tópicos entre Gazebo y ROS2



3.5.2.patricio_nav_punto

3.5.2.1. Estructura del paquete

patricio_nav_punto

- |— launch
 - | |— my_map_server.launch.py
 - | |— my_navigation.launch.py
 - | |— my_tb3_navigator.launch.py
 - | |— prueba_mundo.launch.py
- |— map
 - | |— house_map.pgm
 - | |— house_map.yaml
 - | |— my_map.pgm
 - | |— my_map.yaml
 - | |— prueba_mundo_map.pgm
 - | |— prueba_mundo_map.yaml
- |— package.xml
- |— param
 - | |— burger_cam.yaml
 - | |— burger.yaml
 - | |— turtlebot3_burger.urdf
- |— patricio_nav_punto
 - | |— _init_.py
- |— resource
 - | |— patricio_nav_punto
- |— rviz
 - | |— tb3_navigation2.rviz
- |— setup.cfg
- |— setup.py
- |— test
 - | |— test_copyright.py
 - | |— test_flake8.py
 - | |— test_pep257.py



```
└─ worlds
  └─ prueba_mundo.world
```

3.5.2.2. Descripción

Este paquete se encarga de la **navegación autónoma del robot** utilizando el stack de navegación de ROS 2, concretamente **Nav2 (Navigation2)**.

Permite al robot:

- Localizarse en el mapa
- Planificar rutas
- Evitar obstáculos
- Desplazarse hacia objetivos

El sistema se basa en la carga de mapas (**.yaml** + **.pgm**) y en la ejecución de múltiples nodos coordinados mediante el sistema de navegación de ROS2.

3.5.2.3. Topics

Topic	Tipo de dato	Descripción
/map	nav_msgs/OccupancyGrid	Mapa del entorno
/amcl_pose	geometry_msgs/PoseWithCovarianceStamped	Posición estimada del robot
/cmd_vel	geometry_msgs/Twist	Comandos de velocidad
/goal_pose	geometry_msgs/PoseStamped	Objetivo de navegación
/plan	nav_msgs/Path	Ruta calculada
/scan	sensor_msgs/LaserScan	Datos del LiDAR
/tf	tf2_msgs/TFMessage	Transformaciones dinámicas
/tf_static	tf2_msgs/TFMessage	Transformaciones



		estáticas
--	--	-----------

3.5.2.4. Nodos

Nodo	Paquete	Descripción
map_server	nav2_map_server	Publica el mapa del entorno
amcl	nav2_amcl	Localización probabilística del robot
planner_server	nav2_planner	Calcula rutas hacia el objetivo
controller_server	nav2_controller	Genera comandos de movimiento
bt_navigator	nav2_bt_navigator	Gestiona el comportamiento de navegación
recoveries_server	nav2_recoveries	Gestiona comportamientos de recuperación
lifecycle_manager	nav2_lifecycle_manager	Gestiona el ciclo de vida de los nodos
rviz2	rviz2	Visualización del sistema

3.5.3.patricio_captacion

Estructura del paquete

```
patricio_captacion
├── package.xml
```



```
|— patricio_captacion
|   |— __init__.py
|— resource
|   |— patricio_captacion
|— setup.cfg
|— setup.py
|— test
|   |— test_copyright.py
|   |— test_flake8.py
|   |— test_pep257.py
```

Descripción

El paquete `patricio_captacion` está destinado a la **gestión de la captación de datos sensoriales del robot**, aunque en su estado actual actúa como una estructura base para futuras ampliaciones.

Incluye la configuración básica de un paquete ROS2 en Python, preparado para la implementación de nodos encargados de la adquisición y procesamiento de información del entorno.

Su función dentro del sistema es servir como punto de partida para integrar:

- Sensores físicos
- Datos del entorno
- Entradas del usuario

Este paquete no define y no contiene topics y nodos.

3.5.4.patricio_interfaces

3.5.4.1.Estructura del paquete

```
patricio_interfaces
|— CMakeLists.txt
|— package.xml
|— srv
|   |— IniciarEscondite.srv
|   |— StartGame.srv
```



3.5.4.2.Descripción

El paquete patricio_interfaces define las **interfaces de comunicación mediante servicios** entre los distintos nodos del sistema en ROS 2.

Estos servicios permiten ejecutar acciones concretas, como iniciar juegos o controlar el comportamiento del robot, mediante comunicación síncrona entre nodos.

3.5.4.3.Servicios

3.5.4.3.1. Request

Campo	Tipo	Descripción
poses	geometry_msgs/PoseArray	Lista de posiciones candidatas
command	string	"START" o "STOP"

3.5.4.3.2. Response

Campo	Tipo	Descripción
success	bool	Indica si el comando fue ejecutado correctamente
message	string	Información adicional del resultado
target_pose	geometry_msgs/Pose	Posición objetivo seleccionada (solo en START)

3.5.5.patricio_pilla_pilla

3.5.5.1. Estructura del paquete



```

patricio_pilla_pilla
├── launch
│   └── pilla_pilla.launch.py
├── package.xml
├── patricio_pilla_pilla
│   ├── __init__.py
│   └── pilla_pilla_node.py
├── resource
│   └── patricio_pilla_pilla
├── setup.cfg
├── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py

```

3.5.5.2.Descripción

El paquete `patricio_pilla_pilla` implementa el juego interactivo **“Pilla-Pilla”** como un nodo en ROS 2.

Este nodo permite al robot desplazarse de forma autónoma entre diferentes puntos del mapa, simulando un comportamiento dinámico basado en rutas generadas automáticamente.

El sistema combina navegación, gestión de estados y comunicación mediante servicios y tópicos.

3.5.5.3. Topics

Topic	Tipo de dato	Descripción
<code>/patricio/pilla_pilla/status</code>	<code>std_msgs/String</code>	Estado actual del juego
<code>/goal_pose</code>	<code>geometry_msgs/PoseStamped</code>	Objetivo de navegación
<code>/odom</code>	<code>nav_msgs/Odometry</code>	Posición actual del robot



/patricio/pilla_pilla/cmd	std_msgs/String	Comandos de control (STOP, DETENER)
/goal_pose	geometry_msgs/PoseStamped	Objetivo de navegación
/cmd_vel	geometry_msgs/Twist	Comandos de velocidad
/odom	nav_msgs/Odometry	Posición del robot
/scan	sensor_msgs/LaserScan	Datos del sensor LiDAR
/plan	nav_msgs/Path	Ruta calculada
/amcl_pose	geometry_msgs/PoseWithCovarianceStamped	Posición estimada
/map	nav_msgs/OccupancyGrid	Mapa del entorno
/tf	tf2_msgs/TFMessage	Transformaciones dinámicas
/tf_static	tf2_msgs/TFMessage	Transformaciones estáticas
/clock	roscpp_msgs/Clock	Tiempo de simulación

3.5.5.4.Nodos

Nodo	Paquete	Descripción
pilla_pilla_node	patricio_pilla_pilla	Gestiona la lógica del juego
planner_server	nav2_planner	Calcula rutas hacia los objetivos
controller_server	nav2_controller	Genera comandos de movimiento
bt_navigator	nav2_bt_navigator	Gestiona el comportamiento de navegación
amcl	nav2_amcl	Estima la posición del



		robot en el mapa
map_server	nav2_map_server	Publica el mapa del entorno
lifecycle_manager	nav2_lifecycle_manager	Gestiona el ciclo de vida de los nodos
recoveries_server	nav2_recoveries	Gestiona comportamientos de recuperación
rviz2	rviz2	Visualización del sistema
gz_sim	ros_gz_sim	Ejecuta la simulación en Gazebo
robot_state_publisher	robot_state_publisher	Publica las transformaciones del robot
spawn_entity	ros_gz_sim	Inserta el robot en el entorno
parameter_bridge	ros_gz_bridge	Conecta Gazebo con ROS2

3.5.6.patricio_escondite

3.5.6.1. Estructura del paquete

```

patricio_escondite
├── package.xml
├── patricio_escondite
│   ├── escondite.py
│   ├── escondite_service.py
│   └── __init__.py
├── resource
└── patricio_escondite

```



```
|— setup.cfg
|— setup.py
|— test
  |— test_copyright.py
  |— test_flake8.py
  |— test_pep257.py
```

3.5.6.2. Descripción

El paquete `patricio_escondite` implementa el juego interactivo “**Escondite**”, permitiendo al robot buscar un objetivo dentro de un conjunto de posiciones proporcionadas.

El diseño sigue una arquitectura modular en ROS 2, separando claramente:

- La **interfaz ROS2** (servicios, tópicos)
- La **lógica del juego** (independiente de ROS)

Esto mejora la mantenibilidad, reutilización y claridad del sistema.

El juego de *Escondite* utiliza un conjunto de coordenadas seleccionadas sobre el mapa para definir las posibles posiciones donde el robot debe realizar la búsqueda.

Estas coordenadas se envían desde la interfaz web al sistema mediante la API y son utilizadas posteriormente por el nodo de escondite para generar los objetivos de navegación.

Cada coordenada está definida en el sistema de referencia map y tiene la forma:

(x,y,z)

donde:

- **x** representa la posición en el eje horizontal
- **y** representa la posición en el eje vertical
- **z** representa la altura, siendo en este caso siempre 0.0 al tratarse de navegación en un entorno 2D

Las coordenadas utilizadas actualmente en el sistema son:



- (-1.5, 5.5, 0.0)
- (5.5, 2.0, 0.0)
- (4.0, 4.0, 0.0)

Estas posiciones son seleccionadas manualmente desde la interfaz web y corresponden a zonas navegables dentro del mapa.

Durante la ejecución del juego, el sistema sigue la siguiente lógica:

- Se selecciona una de las coordenadas como objetivo real de forma aleatoria
- El resto de coordenadas se consideran puntos falsos
- El robot recorre primero los puntos falsos y finalmente el objetivo real

Este comportamiento permite simular el juego del escondite, en el que el robot debe “buscar” en diferentes ubicaciones hasta encontrar el objetivo.

3.5.6.3. Topics

Topic	Tipo de dato	Descripción
/patricio/escondite/status	std_msgs/String	Estado del juego
/patricio/escondite/control	std_msgs/String	Control del juego (STOP)
/goal_pose	geometry_msgs/PoseStamped	Objetivo de navegación
/cmd_vel	geometry_msgs/Twist	Comandos de velocidad
/odom	nav_msgs/Odometry	Posición del robot
/scan	sensor_msgs/LaserScan	Datos del sensor LiDAR
/plan	nav_msgs/Path	Ruta calculada



/amcl_pose	geometry_msgs/PoseWithCovarianceStamped	Posición estimada
/map	nav_msgs/OccupancyGrid	Mapa del entorno
/tf	tf2_msgs/TFMessage	Transformaciones dinámicas
/tf_static	tf2_msgs/TFMessage	Transformaciones estáticas
/clock	roscpp_msgs/Clock	Tiempo de simulación

3.5.6.4. Nodos

Nodo	Paquete	Descripción
escondite_service_nodo	patricio_escondite	Gestiona el servicio y la lógica del juego
planner_server	nav2_planner	Calcula rutas hacia el objetivo
controller_server	nav2_controller	Genera comandos de movimiento
bt_navigator	nav2_bt_navigator	Gestiona el comportamiento de navegación
amcl	nav2_amcl	Estima la posición del robot en el mapa
map_server	nav2_map_server	Publica el mapa del entorno
lifecycle_manager	nav2_lifecycle_manager	Gestiona el ciclo de vida de los nodos
recoveries_server	nav2_recoveries	Gestiona comportamientos de recuperación
gz_sim	ros_gz_sim	Ejecuta la simulación en



		Gazebo
robot_state_publisher	robot_state_publisher	Publica las transformaciones del robot
parameter_bridge	ros_gz_bridge	Conecta Gazebo con ROS2

3.5.6.5. Servicios

Servicio	Tipo	Descripción
/patricio/escondite/iniciar	patricio_interfaces/IniciarEscondite	Inicia o detiene el juego



3.5.7.patricio_web

3.5.7.1. Estructura del paquete

patricio_web

- |— admin.html
- |— base_header_footer.html
- |— contacto.html
- |— css
 - | |— styles_admin.css
 - | |— styles_contacto.css
 - | |— styles_empleado.css
 - | |— styles_generales.css
 - | |— styles_index.css
 - | |— styles_login.css
 - | |— styles_logout.css
 - | |— styles_politica_privacidad.css
 - | |— styles_usuario.css
- |— empleado.html
- |— images
 - | |— logo_patricio.png
 - | |— robot_login.png
 - | |— robot.png
- |— index.html
- |— js
 - | |— draw_occupancy_grid.js
 - | |— ros_logic.js
 - | |— script_admin.js
 - | |— script_empleado.js
 - | |— script_header_user.js
 - | |— script_monitor.js
 - | |— script_singup_login_logout.js
 - | |— script_usuario.js
- |— patricio_api.py
- |— politica_privacidad.html
- |— static



```
| └─ arrancar_web.sh
└─ usuario.html
```

3.5.7.2. Descripción

El paquete `patricio_web` implementa una **interfaz web de control remoto** para el robot Patricio.

Permite al usuario interactuar con el sistema en tiempo real mediante una arquitectura basada en:

- Frontend web (HTML, CSS, JavaScript)
- Backend en Flask (API REST)
- Comunicación con ROS2 mediante **rosbridge**

Esto permite controlar el robot desde un navegador sin necesidad de ejecutar comandos ROS directamente.

3.5.7.3. Topics (usados desde la web)

Topic	Tipo de dato	Dirección	Descripción
/patricio/pilla_pilla/cmd	std_msgs/String	Publica	Envía STOP al juego
/patricio/pilla_pilla/status	std_msgs/String	Suscribe	Estado del juego pilla-pilla
/patricio/escondite/status	std_msgs/String	Suscribe	Estado del juego escondite

3.5.7.4. Servicios

Servicio	Tipo	Descripción
/start_game	patricio_interfaces/Start Game	Inicia pilla-pilla
/patricio/escondite/iniciar		Inicia o detiene escondite



4. Mapa y navegación

4.1. Mapa

La imagen representa la disposición del entorno interior de una habitación de un hogar, utilizada como mapa de navegación para el robot Patricio durante las tareas de localización, educación y entretenimiento del niño. Este entorno ha sido digitalizado y preparado para su uso en simulaciones y en la planificación de rutas autónomas con ROS 2 y Nav2.

4.2. Estructura del Mapa

Fondo gris claro: representa el área fuera del entorno de navegación del robot (muros, zonas no transitables).

Zona blanca: corresponde al espacio transitable por el robot, es decir, el suelo libre donde Patricio puede desplazarse.

Zona negra: corresponde al espacio ocupado por los diferentes muebles y estructuras situadas dentro del entorno.

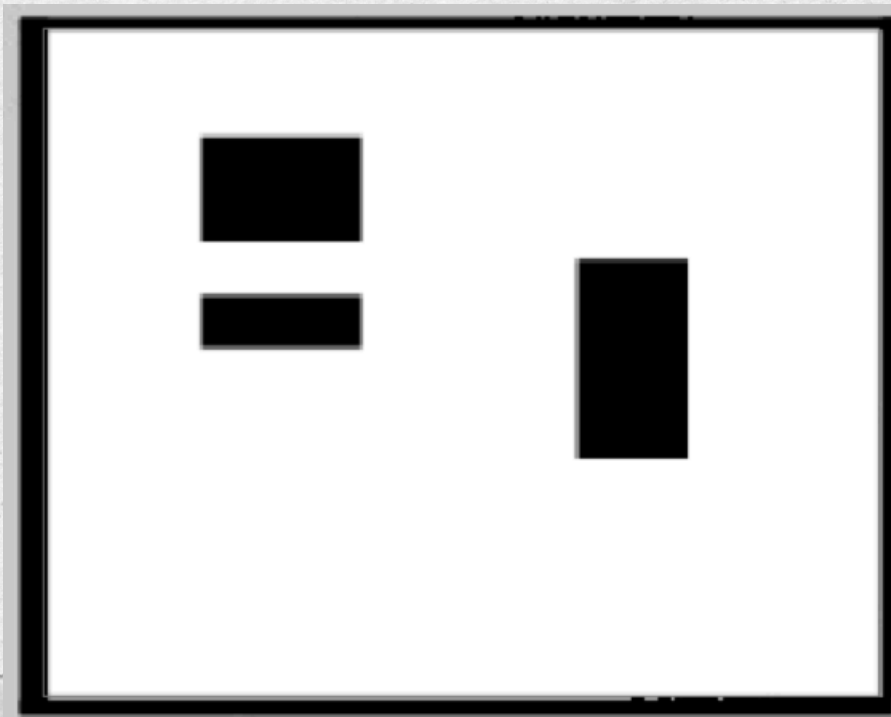


Figura 1.



4.2.1. Uso por Patricio

El mapa ha sido configurado como parte del entorno simulado en Gazebo y está asociado a la navegación autónoma del robot Patricio. Gracias a la información obtenida de sensores láser y técnicas de localización como AMCL, el robot puede:

Navegar evitando obstáculos.

Identificar el mostrador como punto de interacción con el personal.

Acercarse a las estanterías con precisión para recoger o entregar medicamentos. Seguir rutas planificadas dentro de un espacio estructurado y funcional.

5. Diseño Backend

5.1. Base de Datos

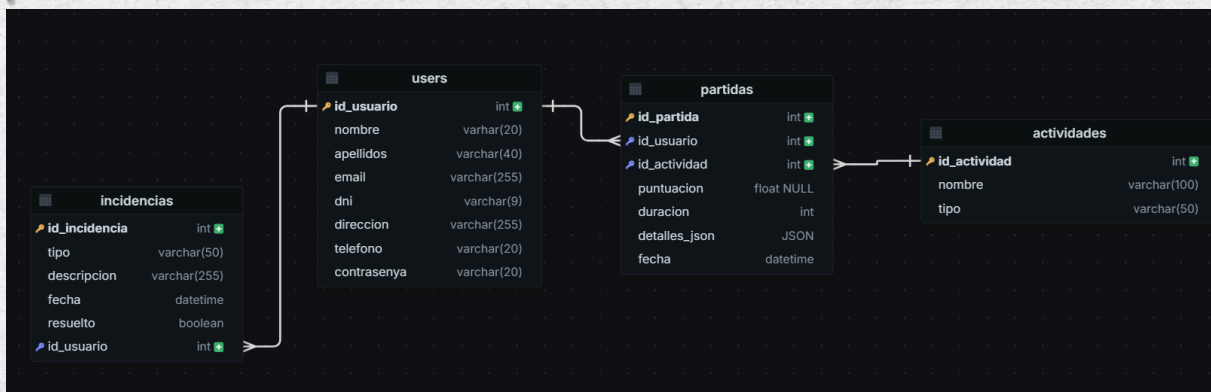


Figura 2.

La base de datos está pensada para gestionar la información de un sistema que cubre:

- Usuarios
- Actividades
- Partidas
- Incidencias

La entidad central es **Usuarios**, donde se guardan los datos personales de los padres: nombre, apellidos, DNI, dirección, correo electrónico, teléfono y contraseña para el acceso al sistema. Cada usuario queda identificado mediante



id_usuario, que se utiliza para relacionarlo con el resto de tablas.

Las **Actividades** describen el conjunto de acciones que puede realizar el robot, como juegos (pilla-pilla, escondite) o actividades educativas (mates simples, lectura de cuentos). Cada actividad se identifica mediante id_actividad y contiene información como el nombre y el tipo, que permite clasificarla en categorías como matemáticas, lectura o movimiento.

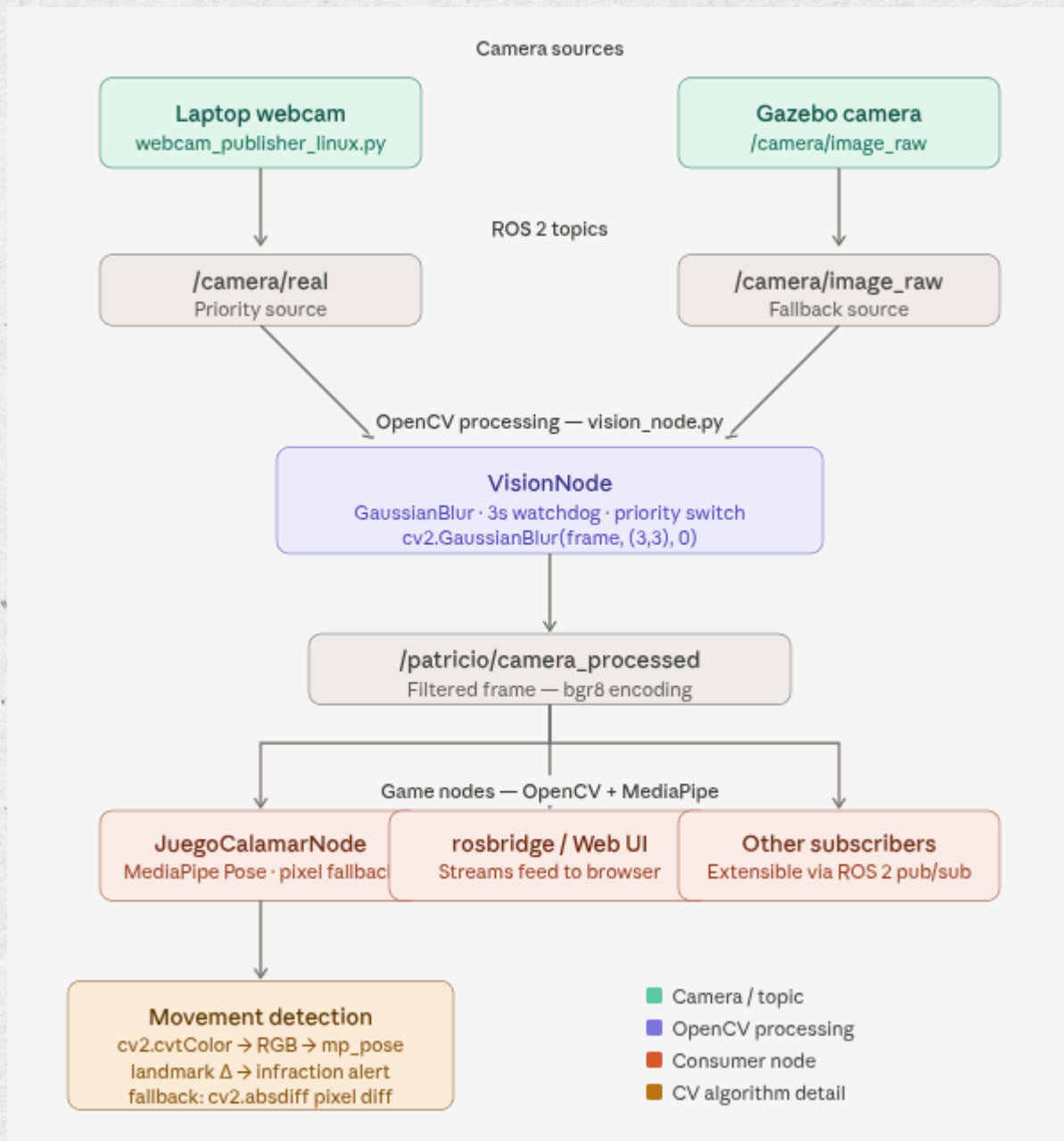
Las interacciones del sistema se registran en la tabla **Partidas**. Cada fila representa una ejecución concreta de una actividad por parte de un usuario, identificada mediante id_partida. Esta tabla se relaciona con Usuarios mediante id_usuario y con Actividades mediante id_actividad. Además, incluye información como la puntuación (cuando la actividad es evaluable), la duración, la fecha y un campo detalle_json, que permite almacenar información específica de cada actividad (por ejemplo, aciertos y errores en matemáticas o datos de uso en juegos y cuentos).

La tabla **Incidencias** permite registrar anomalías detectadas por el robot, como caídas, batería baja u otros errores. Cada incidencia se identifica mediante id_incidencia y puede incluir el tipo, una descripción, la fecha y su estado (resuelto). También puede estar asociada a un usuario mediante id_usuario, lo que permite saber en qué contexto ocurrió la incidencia.

En conjunto, la estructura permite gestionar los usuarios del sistema, definir las actividades disponibles, registrar el historial de uso del robot y controlar incidencias detectadas durante su funcionamiento. Además, el uso de un campo JSON en la tabla de partidas aporta flexibilidad, permitiendo adaptar el sistema a diferentes tipos de actividades sin modificar la estructura de la base de datos.



5.2. Lógica de visión artificial



6. Stack Tecnológico

El proyecto Patricio se apoya en un conjunto de tecnologías abiertas y estandarizadas, seleccionadas para garantizar la portabilidad, la mantenibilidad y la reutilización académica del trabajo realizado. A continuación se justifica cada una de las decisiones tecnológicas adoptadas, agrupadas según la capa de la arquitectura a la que pertenecen.



6.1. Capa de robot (back-end robótico)

ROS 2 Jazzy Jalisco es el middleware de robótica sobre el que se construye todo el sistema. Se ha seleccionado por ser la versión LTS más reciente (soporte 2024-2029), por su modelo de publicador/suscriptor basado en tópicos que encaja perfectamente con la arquitectura modular del proyecto, y por contar con una comunidad activa y abundante documentación.

Gazebo es el simulador 3D que permite probar el comportamiento del robot en un entorno virtual realista antes de pasar al despliegue físico. Se integra de forma nativa con ROS 2 mediante el puente `ros_gz_bridge` y soporta la simulación del modelo TurtleBot3 Burger utilizado en el proyecto. El entorno doméstico (`house.sdf`) reproduce una vivienda con mobiliario realista para que las pruebas de navegación reflejen el caso de uso real del producto.

Nav2 es el stack oficial de navegación autónoma de ROS 2. Aporta los componentes necesarios para que Patricio pueda localizarse mediante AMCL, planificar rutas con `planner_server`, ejecutar movimientos mediante `controller_server` y gestionar comportamientos de recuperación, todo ello a partir de un mapa de ocupación previamente generado. El proyecto utiliza dos modos de navegación complementarios: navegación a un punto único (paquete `patricio_nav_punto`) y planificación y seguimiento de rutas (paquete `patricio_nav_ruta`).

OpenCV es la librería de visión por computador empleada para el procesamiento de la imagen captada por la cámara del robot. Se ha optado por OpenCV por ser el estándar de facto en visión artificial sobre Python, por su excelente rendimiento en tiempo real y por la disponibilidad de algoritmos preparados tanto para detección de personas como para detección de movimiento, ambas funcionalidades clave en el caso del juego del Calamar.

Python 3 es el lenguaje principal de los nodos ROS 2 propios del proyecto, escogido por su rapidez de desarrollo, la disponibilidad de bibliotecas científicas y por integrarse de forma nativa con ROS 2 mediante `rclpy`. Se complementa con **C++** únicamente en aquellos componentes donde el rendimiento lo justifica (por ejemplo, los nodos heredados del paquete TurtleBot3, que representan aproximadamente el 7% del código del proyecto).

6.2. Capa de comunicación (back-end web)

Flask (Python) es el microframework con el que se ha implementado la API REST del back-end (`api.py`). Se ha elegido por su ligereza, por su curva de aprendizaje suave y por la coherencia con el resto de la pila tecnológica, también basada en Python. Flask gestiona la autenticación de usuarios, las sesiones, el registro del historial de partidas y la comunicación con la base de datos.

MySQL es el sistema de gestión de bases de datos relacional adoptado para la persistencia de la información del sistema (usuarios, actividades, partidas e incidencias). Se ha optado por una base de datos relacional frente a una solución NoSQL porque los datos del proyecto presentan relaciones claras entre entidades (un usuario tiene muchas partidas, una partida



pertenece a una actividad, etc.) que se modelan de forma natural con claves foráneas, y porque MySQL es un estándar maduro, ampliamente documentado y soportado por cualquier hosting.

Rosbridge Suite es el componente que actúa como puente entre el mundo web y el mundo ROS 2. Expone un servidor WebSocket en el puerto 9090 que traduce de forma bidireccional los mensajes JSON enviados desde el navegador a mensajes ROS 2 nativos, y viceversa. Su uso permite que el front-end no necesite tener ROS 2 instalado y que la web se pueda servir desde cualquier dispositivo con un navegador moderno.

WebSockets se ha seleccionado como protocolo de comunicación principal entre el navegador y el robot frente a HTTP convencional porque mantiene un canal abierto y bidireccional de forma permanente. Esto es imprescindible para el control en tiempo real del robot, donde tanto los comandos del usuario como el feedback del robot deben fluir de forma continua y con latencia mínima.

Servidor de vídeo MJPEG sobre HTTP. Se ha implementado un servidor de vídeo independiente del back-end principal para servir el flujo de la cámara del robot al navegador. Se ha optado por el formato MJPEG por su simplicidad, su soporte universal en navegadores sin necesidad de plugins, y su baja latencia frente a alternativas más complejas como WebRTC, suficiente para el caso de uso del proyecto.

6.3. Capa de front-end

HTML5, CSS3 y JavaScript son las tecnologías base de la interfaz web. Suponen conjuntamente cerca del 68% del código del proyecto, lo que refleja el peso que tiene la interfaz de usuario en el sistema. Se ha optado por JavaScript sin frameworks (vanilla) para evitar dependencias innecesarias, simplificar el despliegue y reducir el peso de la página, dada la naturaleza relativamente sencilla del front-end del proyecto.

roslibjs es la biblioteca JavaScript oficial para comunicarse con Rosbridge desde el navegador. Permite publicar y suscribirse a tópicos ROS 2 directamente desde el código JavaScript del front-end, abstrayendo la serialización JSON y la gestión del WebSocket.

ros2djs se utiliza para la visualización 2D del mapa de ocupación del robot directamente en el navegador, complementada con el script propio [draw_occupancy_grid.js](#).

DM Sans es la familia tipográfica adoptada para toda la interfaz, en coherencia con la identidad visual definida en el moodboard del proyecto. La paleta de colores corporativos (amarillo F2D88F, naranja F08C21, rosa E36888, azul 6698CC, verde B4B534 y crema FFF4D6) acompaña la tipografía para reforzar la identidad visual y orientar emocionalmente al público infantil al que se dirige el producto.

6.4. Pantalla de caras del robot

La pantalla integrada del robot se implementa como una página web independiente ([face_screen.html](#)) que también se ejecuta sobre **HTML5, CSS3 y JavaScript**, y que se comunica con el sistema ROS 2 a través del mismo Rosbridge que el resto de la interfaz.

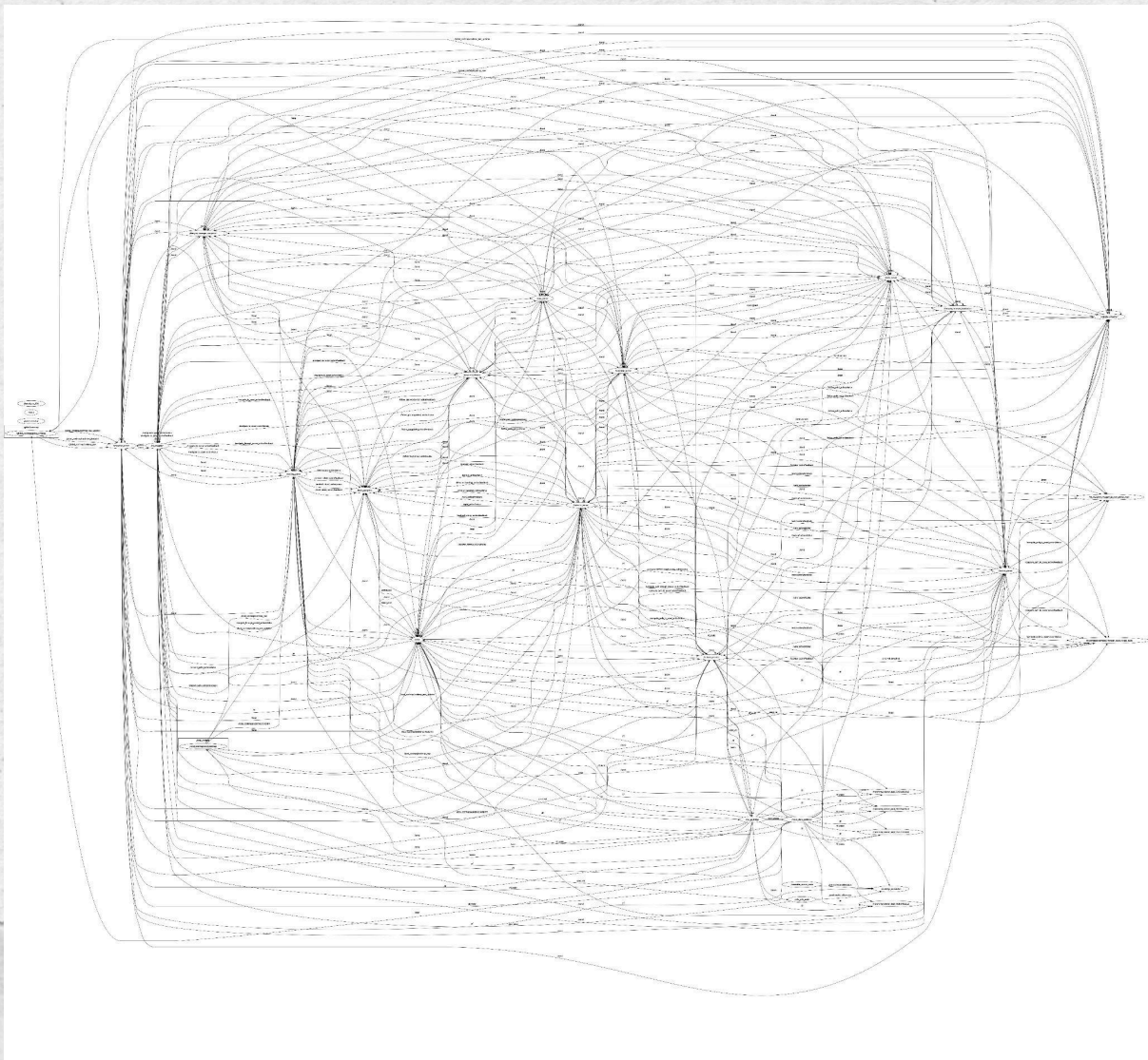


De esta forma, el rostro del robot reacciona en tiempo real a los cambios de estado y de actividad publicados en los tópicos `/patricio/face_status` y `/patricio/game_status`.

6.5. Sistema operativo y herramientas auxiliares

El sistema se ejecuta sobre **Ubuntu 24.04 LTS (Noble Numbat)**, sistema operativo recomendado para ROS 2 Jazzy. El control de versiones del proyecto se realiza con **Git** y el repositorio se aloja en **GitHub** (github.com/CJMIDNIGHT/patricio), lo que ha permitido al equipo trabajar en paralelo sobre los distintos paquetes sin conflictos. La compilación del workspace se realiza con **colcon**, herramienta estándar de construcción de paquetes ROS 2, y la gestión de dependencias con **rosdep**.

7. Nodos



El siguiente diagrama representa la arquitectura de nodos del sistema ROS 2 del robot Patricio, mostrando la red de comunicaciones entre los distintos componentes durante su ejecución en un entorno simulado con Gazebo y el sistema de navegación Nav2.

Este grafo, generado mediante `rqt_graph`, permite visualizar los nodos activos, así como los tópicos y servicios a través de los cuales se comunican.

Componentes clave:

Nodos de navegación (Nav2):

Los nodos `bt_navigator`, `planner_server`, `controller_server`, `map_server` y `amcl` constituyen el núcleo del sistema de navegación autónoma.

Se encargan de la planificación de rutas, el control del movimiento y la localización del robot dentro del mapa.

Transformaciones y posicionamiento:

Los nodos `robot_state_publisher` y el sistema `tf` gestionan las transformaciones entre los distintos sistemas de referencia (`map`, `odom`, `base_link`).

La comunicación se realiza principalmente a través de los tópicos `/tf` y `/tf_static`.

Simulación del entorno:

El entorno se ejecuta en Gazebo mediante el nodo `gz_sim`.

El nodo `parameter_bridge` permite la comunicación entre Gazebo y ROS 2.

Se utilizan tópicos como `/odom` (odometría), `/scan` (sensor LiDAR) y `/cmd_vel` (control de movimiento).

Nodos propios del proyecto:

Se incluyen nodos desarrollados específicamente para el robot Patricio, como `pilla_pilla_node` y `escondite_service_node`.

Estos implementan los juegos interactivos y utilizan el sistema de navegación para moverse por el entorno.

Interfaz web:

El sistema dispone de una interfaz web que permite controlar el robot de forma remota.

Se comunica mediante tópicos como `/patricio/pilla_pilla/cmd` y `/patricio/pilla_pilla/status`, y servicios como `/start_game` y `/patricio/escondite/iniciar`.



Publicadores y suscriptores:

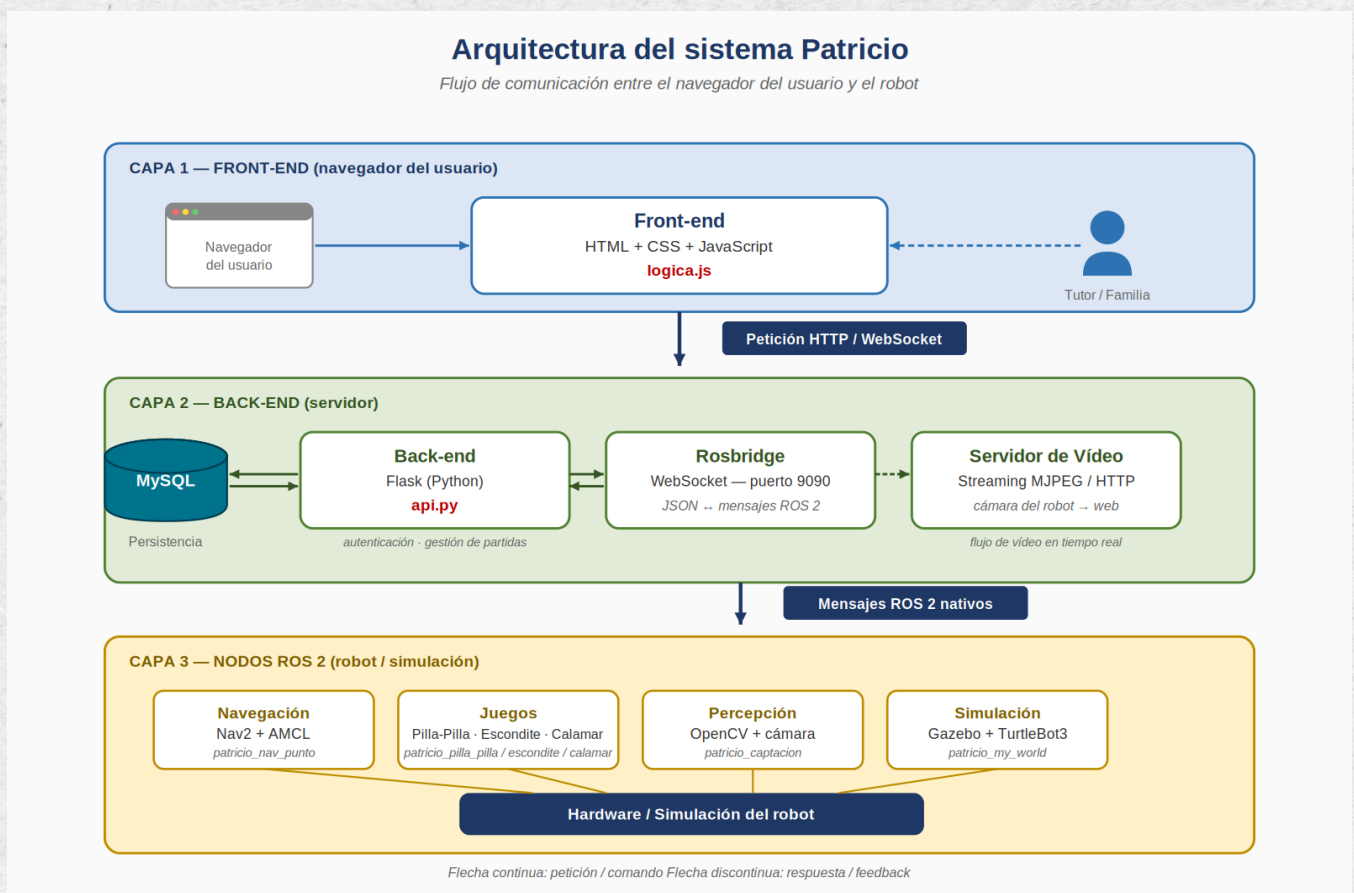
Entre los principales tópicos destacan `/cmd_vel`, `/odom`, `/scan` y `/goal_pose`, utilizados para el control del robot, la percepción del entorno y la navegación.

Conclusión:

El diagrama muestra cómo los nodos propios del proyecto se integran con los componentes estándar de ROS 2, permitiendo combinar navegación autónoma, interacción mediante juegos y control remoto a través de la interfaz web.

8. Arquitectura del sistema

8.1. Diagrama de la Arquitectura del sistema



Este diagrama representa el flujo completo de datos del proyecto Patricio, desde que un operador pulsa un botón en el navegador hasta que el robot ejecuta una acción en el simulador. La arquitectura está dividida en tres capas bien diferenciadas, cada una con una responsabilidad clara y unos componentes específicos.



8.1.1.Capa 1 — Usuario

BLOQUES AZULES

Esta capa es todo lo que el operador ve y toca. El navegador accede a http://<IP_DEL_ROBOT>:8080 donde se sirve la interfaz del paquete patricio_web. La web no es una página única sino un sistema completo con **tres roles de usuario** diferenciados: admin.html para el administrador del sistema, empleado.html para el operador del robot, y usuario.html para el usuario final. Cada rol tiene su propia hoja de estilos CSS y su propio script JavaScript (script_admin.js, script_empleado.js, script_usuario.js), lo que garantiza que cada perfil solo ve y puede hacer lo que le corresponde.

Además de los scripts de rol, hay scripts transversales igualmente importantes: script_monitor.js gestiona la monitorización del estado del robot en tiempo real, y draw_occupancy_grid.js es especialmente relevante porque se encarga de dibujar el mapa de ocupación del entorno directamente en el navegador, mostrando al operador dónde está el robot y qué obstáculos hay a su alrededor.

La librería **roslibjs** es el elemento técnico clave de esta capa. Se eligió porque permite que el navegador publique y se suscriba a tópicos ROS 2 sin que el operador tenga ROS instalado, lo cual es fundamental en un entorno como un aula donde los usuarios no tienen perfil técnico. Todo lo que sale de esta capa viaja serializado en formato **JSON**.

8.1.1.1.Flecha — WebSocket, puerto 9090

FLECHAS ENTRE BLOQUES AZULES Y VERDES

La comunicación entre el navegador y el robot usa **WebSocket** en lugar de HTTP convencional porque WebSocket mantiene un canal abierto y bidireccional de forma permanente. Esto es imprescindible para controlar un robot en tiempo real: la flecha sólida representa los comandos bajando al robot, y la flecha discontinua el feedback del robot subiendo continuamente al navegador. Todo circula por el **puerto 9090**, el puerto estándar de rosbridge en ROS 2.



8.1.2.Capa 2 — Comunicación

BLOQUES VERDES

Aquí vive el nodo **rosbridge_server**, que hace de traductor universal. Recibe el JSON del navegador y lo convierte en mensajes ROS 2 nativos publicándolos en el tópico correspondiente. En sentido contrario, serializa los datos que publican los nodos ROS y los devuelve al navegador. Se eligió esta arquitectura de puente dedicado porque desacopla completamente el frontend del backend: se puede modificar la web sin tocar ningún nodo ROS, y añadir nuevos nodos ROS sin tocar la web.

8.1.2.1.Flecha — Mensajes ROS 2

FLECHAS ENTRE BLOQUES VERDES Y BLOQUES MORADOS

A partir de aquí los datos son mensajes ROS 2 nativos. Se usan tres mecanismos según la naturaleza de cada comunicación: **topics** para datos continuos como velocidad o posición, **actions** para tareas largas con progreso y cancelación como navegar a un punto, y **services** para operaciones puntuales con respuesta confirmada, como arrancar el juego mediante StartGame.srv.

8.1.3.Capa 3 — Robot

Apartado de BLOQUES MORADOS

Aquí reside toda la inteligencia del robot, organizada en módulos con responsabilidades bien separadas.

El módulo **patricio_nav_punto** es el núcleo de navegación. Contiene cuatro launch files para distintos escenarios: `my_navigation.launch.py` lanza la navegación completa con Nav2, `my_map_server.launch.py` arranca únicamente el servidor de mapas, `my_tb3_navigator.launch.py` lanza el navegador específico del TurtleBot3, y `prueba_mundo.launch.py` para entornos de prueba. Dispone



además de tres mapas reales ya contruidos: house_map, my_map y prueba_mundo_map, lo que indica que el robot ha sido probado en entornos distintos. La navegación se apoya en el stack **Nav2** con planner_server para calcular rutas, controller_server para ejecutarlas, amcl para la localización probabilística usando el LiDAR, y map_server para cargar y servir el mapa. Los tópicos principales son /cmd_vel, /odom y /map, y la action /navigate_to_pose.

El módulo **patricio_my_world** lanza el entorno completo de simulación. Contiene el mundo virtual house.sdf que representa la casa donde opera el robot, los modelos 3D del entorno como mesas, sofás, televisor y muebles, y el modelo del propio TurtleBot3 en sus dos variantes: con cámara (turtlebot3_burger_cam) y sin cámara (turtlebot3_burger). El modelo **kumaeye** es la representación 3D de la expresión facial del robot, que aparece en la pantalla integrada de Patricio. El arranque se centraliza en house.launch.py, que también lanza robot_state_publisher para publicar el estado de las articulaciones y spawn_turtlebot3 para instanciar el robot en el simulador.

El módulo **patricio_captacion** es el paquete de percepción sensorial del robot. Aunque en esta versión del repositorio su nodo principal aún está en desarrollo, es el responsable de procesar los datos que llegan de los sensores: el LiDAR publica en /scan y la cámara en /camera/image.

El módulo **patricio_pilla_pilla** implementa el juego de pillapilla autónomo a través de pilla_pilla_node.py. Se arranca con pilla_pilla.launch.py y usa el servicio personalizado **StartGame.srv**, definido en patricio_interfaces, para recibir la orden de inicio del juego desde la web. Este servicio es la interfaz entre el botón que pulsa el operador en el navegador y el comportamiento autónomo del robot.

El módulo **patricio_escondite** implementa el juego del escondite autónomo a través de escondite_node.py. Se arranca con escondite.launch.py y usa el servicio personalizado **StartGame.srv**, definido en patricio_interfaces, para recibir la orden de inicio del juego desde la web. Este servicio es la interfaz entre el botón que pulsa el operador en el navegador y el comportamiento autónomo del robot.



El módulo **patricio_calamar** implementa el juego del escondite autónomo a través de `calamar_node.py`. Se arranca `conescondite.launch.py` y usa el servicio personalizado **StartGame.srv**, definido en `patricio_interfaces`, para recibir la orden de inicio del juego desde la web. Este servicio es la interfaz entre el botón que pulsa el operador en el navegador y el comportamiento autónomo del robot.

patricio_interfaces merece mención especial porque es el paquete que define los tipos de mensajes personalizados del proyecto. Su archivo `StartGame.srv` define el contrato de comunicación del juego: qué datos se envían para arrancar una partida y qué respuesta devuelve el sistema. Sin este paquete, el resto de nodos no podrían comunicarse con tipos de datos propios del proyecto.

En la base de todo está **Gazebo**, que simula la física del entorno `house.world`, genera los datos de los sensores virtuales del TurtleBot3 y ejecuta los comandos de movimiento. Los parámetros de comunicación entre Gazebo y ROS 2 se configuran en `turtlebot3_burger_bridge.yaml` y `turtlebot3_burger_cam_bridge.yaml`.

Flujo de comunicación completo

El operador, con su rol correspondiente, abre la interfaz en el navegador → el script de su rol (`script_empleado.js` o `script_admin.js`) captura el evento → `roslibjs` serializa

el comando en JSON → `WebSocket` lo transmite por el puerto 9090 → `rosbridge_server` lo convierte en un mensaje ROS 2 nativo y lo publica en el tópico correspondiente → si es navegación, `my_navigation.launch.py` activa `Nav2` y se lanza la acción `/navigate_to_pose` → `planner_server` calcula la ruta usando el mapa cargado → `controller_server` la ejecuta enviando velocidades por `/cmd_vel` → si es el juego, se llama al servicio `StartGame.srv` y `pilla_pilla_node.py` toma el control del robot → Gazebo simula el movimiento físico del TurtleBot3 → los sensores virtuales generan nuevos datos en `/scan` y `/odom` → esos datos suben de vuelta por `rosbridge` al navegador → `script_monitor.js` y `draw_occupancy_grid.js` actualizan la pantalla mostrando la



posición del robot en el mapa en tiempo real, cerrando el ciclo de control.

#ffff00

9. Interfaz Web del sistema

La aplicación web desarrollada permite interactuar con el robot Patricio de forma remota mediante una interfaz intuitiva, visual y orientada a usuarios no técnicos (entorno educativo).

9.1. Identidad visual (Moodboard)



Antes de abordar el diseño concreto de cada pantalla, el equipo definió la **identidad visual del producto** mediante un moodboard que recoge las referencias estéticas, la paleta de colores y la tipografía oficial del proyecto. Esta fase previa fue clave para garantizar que todas las pantallas mantuvieran una



coherencia visual entre sí y que el aspecto final transmitiera adecuadamente los valores del producto: cercanía, calidez y orientación al público infantil temprano.

9.2.Referencias visuales

El moodboard reúne imágenes de niños interactuando con juguetes y robots educativos, ilustraciones infantiles tipo flat design y un mascotismo (carácter visual con cara expresiva tipo emoji) que sirve como inspiración directa para el diseño de las expresiones faciales que muestra Patricio en su pantalla integrada. La combinación busca un equilibrio entre **lo lúdico** (formas redondeadas, paleta cálida, ilustraciones amables) y **lo educativo** (orden visual, jerarquías claras, ausencia de elementos distractores).

9.3.Paleta de colores

La paleta oficial del proyecto se compone de seis colores, seleccionados para combinar tonos cálidos —que evocan amabilidad y cercanía con el público infantil— con dos acentos fríos que aportan equilibrio y se utilizan en elementos funcionales:

Color	Código HEX	Uso Principal
Amarillo Claro	#F2D88F	Tarjetas, fondos suaves
Naranja Vivo	#F08C21	Botones principales, cabecera
Rosa Coral	#E36888	Pie de página, llamadas a la acción secundarias
Azul Claro	#6698CC	Bloques de información, paneles
Verde Oliva	#B4B534	Acentos puntuales
Crema	#FFF4D6	Fondos generales

9.4.Tipografía

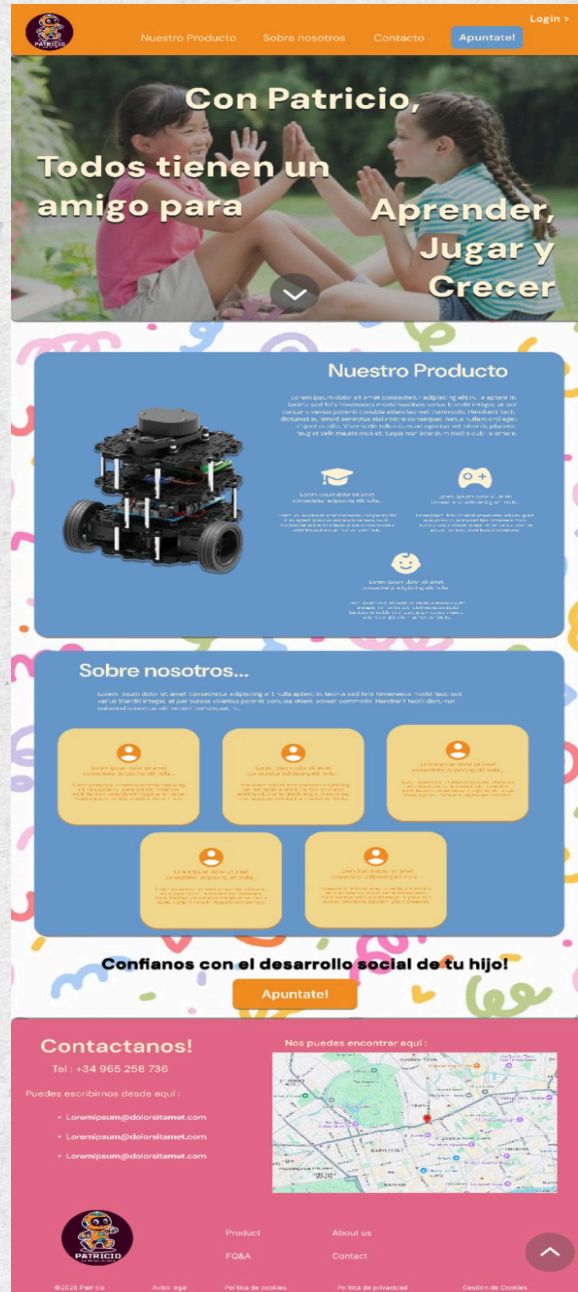


La tipografía oficial del proyecto es **DM Sans**, una familia sans-serif de código abierto desarrollada por Google Fonts. Se ha elegido por tres motivos: ofrece una excelente legibilidad incluso en tamaños pequeños, transmite cercanía gracias a sus terminaciones redondeadas y dispone de múltiples pesos (regular, medium, bold) que permiten establecer jerarquías visuales sin recurrir a una segunda familia tipográfica. Esta elección se aplica de forma consistente en toda la interfaz web, los formularios y los indicadores visuales del robot.



9.5.Pantallas

9.5.1.Landing page - Inicio



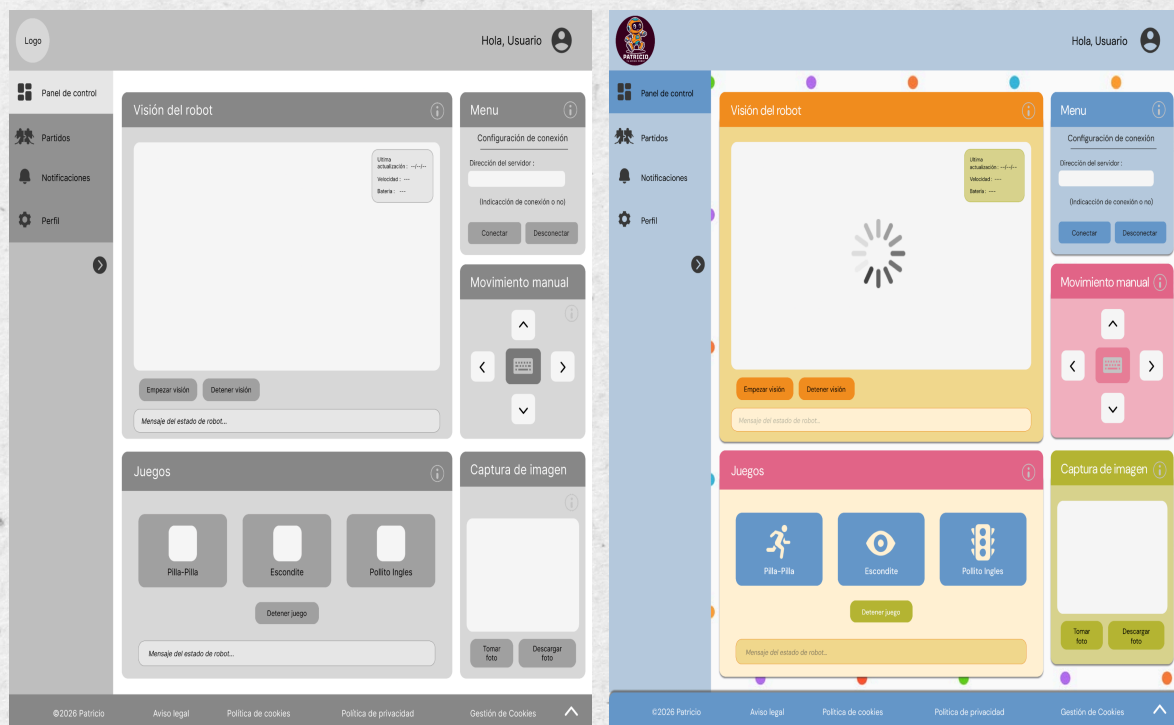
9.5.1.1.Características:

- Diseño visual atractivo con identidad del proyecto
- Feedback inmediato al usuario
- Acceso restringido mediante autenticación



9.5.2. Panel de control (Dashboard)

Pantalla principal tras iniciar sesión, donde se centralizan todas las funcionalidades del sistema.



9.5.3. Panel de juegos

- Selección de modos:
 - **Pilla-Pilla** (modo persecución por waypoints)
 - **Escondite** (modo búsqueda autónoma)
- Botón para **detener juego**
- Indicador dinámico de estado:
 - Esperando instrucciones
 - Ejecutando juego
 - Buscando / moviéndose

9.5.3.1. Configuración de conexión

- Introducción de dirección del servidor (WebSocket)
- Botones:
 - Conectar
 - Desconectar
- Indicador de estado:
 - Conectado / Sin conexión

Restricción importante:

Si no existe conexión activa con el robot, las funcionalidades quedan bloqueadas y se muestra un aviso al usuario.

9.5.3.2. Controles de movimiento

9.5.3.2.1. Movimiento básico:

- Mover
- Parar
- Cambiar sentido

9.5.3.2.2. Control manual (WSAD):

- Adelante
- Atrás



- Izquierda
 - Derecha
 - Parar
-

9.5.4. Monitor del robot

Panel informativo con datos en tiempo real:

- Batería
- Velocidad lineal
- Temperatura
- Última actualización

(Si no hay conexión → datos no disponibles)

9.5.5. Navegación

- Envío del robot a posiciones predefinidas:
 - Zona de espera
 - Base / inicio
 - Taller / material
 - Cancelación de navegación activa
-

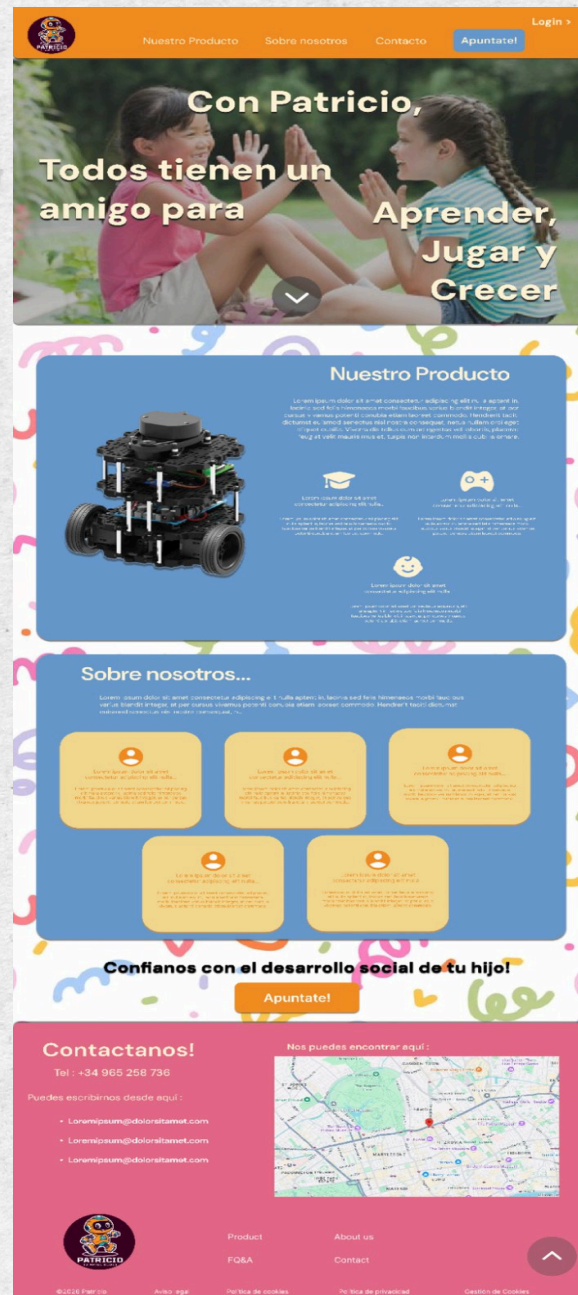
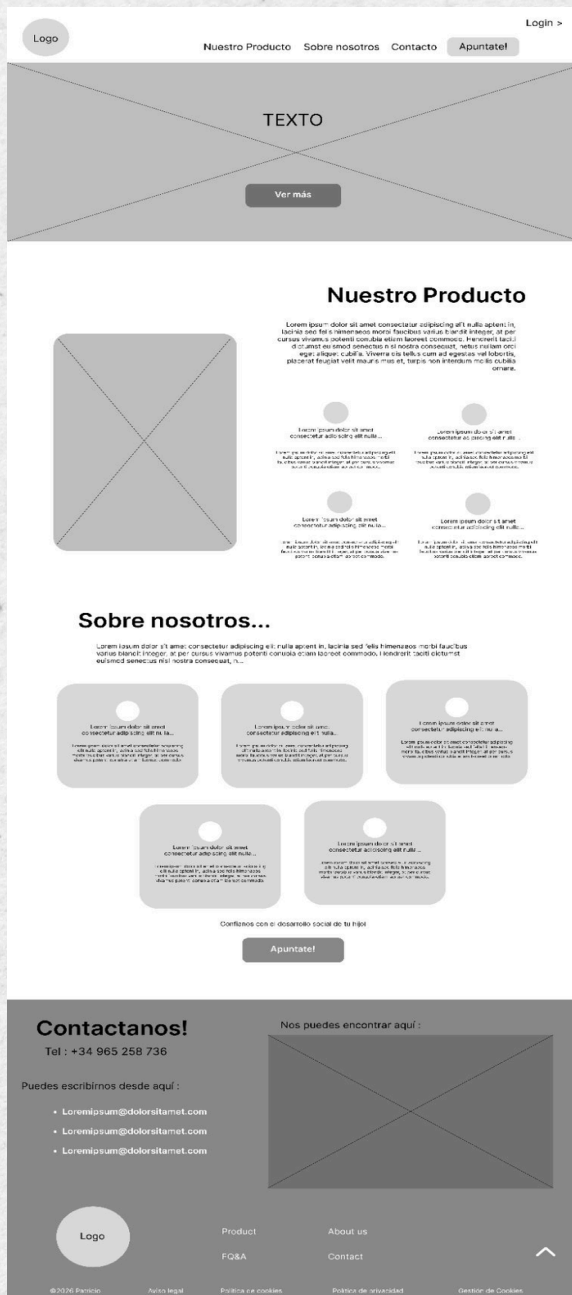
9.5.6. Visualización

- Vista de cámara del robot
 - Vista del mapa
 - Captura de imágenes
-



9.5.7.Sección de presentación (Hero Section)

La parte superior de la landing page corresponde a la **sección de presentación del sistema**, cuyo objetivo es introducir al usuario en el proyecto.



Contenido:

- Nombre del sistema (**Patricio**)
- Descripción breve del robot y su finalidad educativa
- Imagen representativa del robot
- Botón de **registro**
- Enlace a **inicio de sesión**

Función:

- Servir como punto de entrada al sistema
- Explicar de forma clara el propósito del proyecto
- Guiar al usuario hacia la autenticación

9.5.8. Formulario de registro

El sistema incluye un formulario de registro accesible desde la landing page (pulsando el logo) mediante una ventana modal, permitiendo la creación de nuevas cuentas de usuario.

< Volver atrás



Registro

Datos primarios

* 1er Apellido :
Apellido...

* Nombre :
Nombre...

* DNI/INE :
XXXXXXXXXX

* 2o Apellido :
Apellido...

Datos secundarios

* Dirección :
Calle/Avda/Plaza, Pte. Pte., Código postal, Ciudad, País...

* Correo :
nombre@dominio.com...

* Teléfono :
+34 600 000 000...

Datos del usuario

* Usuario :
nombre@dominio.com...

* Contraseña :
+34 600 000 000...

* Confirma Contraseña :
+34 600 000 000...

Regístrate

Ya tienes cuenta? [Inicia sesión!](#)

©2026 Patricio Aviso legal Política de cookies Política de privacidad Gestión de Cookies

< Volver atrás



Registro

Datos primarios

* 1er Apellido :
Apellido...

* Nombre :
Nombre...

* DNI/INE :
XXXXXXXXXX

* 2o Apellido :
Apellido...

Datos secundarios

* Dirección :
Calle/Avda/Plaza, Pte. Pte., Código postal, Ciudad, País...

* Correo :
nombre@dominio.com...

* Teléfono :
+34 600 000 000...

Datos del usuario

* Usuario :
nombre@dominio.com...

* Contraseña :
+34 600 000 000...

* Confirma Contraseña :
+34 600 000 000...

Regístrate

Ya tienes cuenta? [Inicia sesión!](#)

©2026 Patricio Aviso legal Política de cookies Política de privacidad Gestión de Cookies



9.5.8.1. Campos del formulario

- **Nombre**
 - **Apellido**
 - **Email**
 - **Código de grupo o aula**
 - **Teléfono** (opcional)
 - **Contraseña**
 - **Repetición de contraseña**
-

9.5.8.2. Validaciones implementadas

El formulario incluye validaciones para garantizar la correcta introducción de datos:

- Campos obligatorios marcados (*)
 - Contraseña con requisitos mínimos:
 - Mínimo 8 caracteres
 - Al menos una letra mayúscula
 - Al menos una letra minúscula
 - Al menos un número
 - Coincidencia entre contraseña y repetición
-

9.5.8.2.1. Funcionalidad

- Permite registrar nuevos usuarios en el sistema
 - Asocia al usuario con un grupo o aula
 - Prepara al usuario para el acceso al panel de control
-

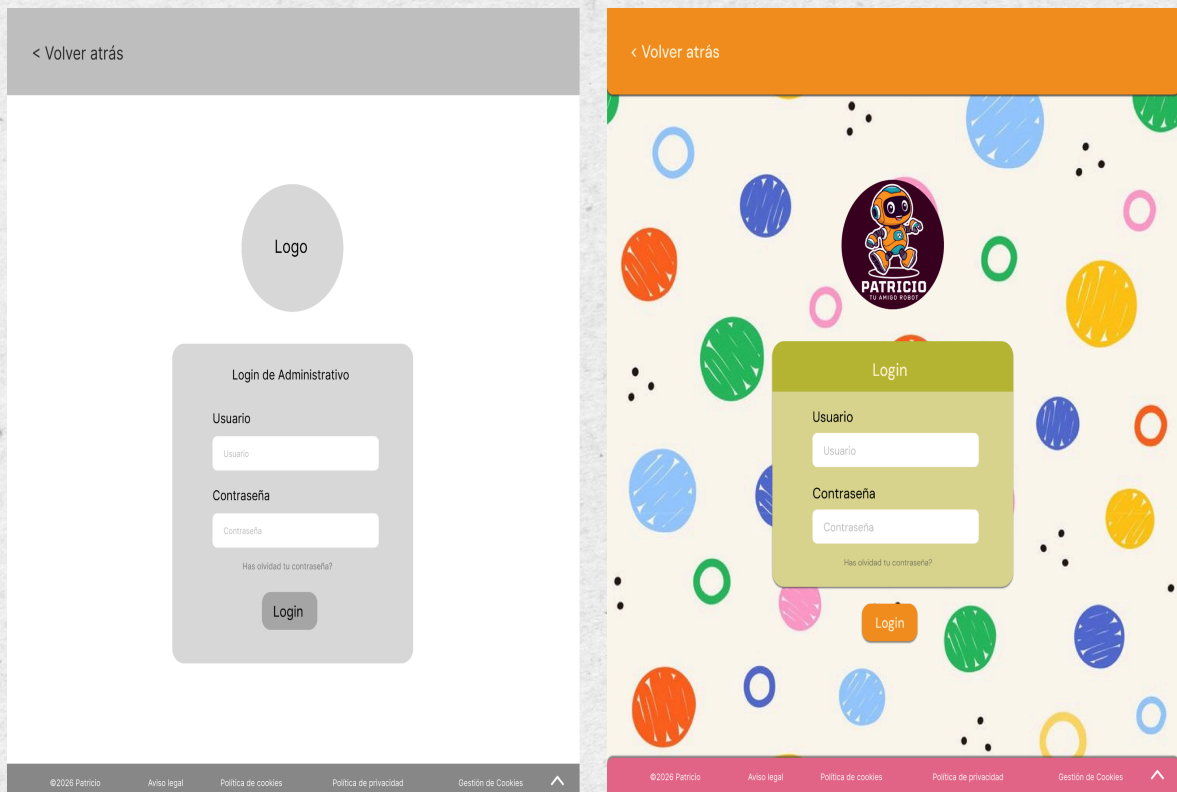
9.5.8.2.2. Características de diseño

- Implementado como **ventana modal** (no cambia de página)
- Interfaz clara y estructurada



- Feedback visual de requisitos de contraseña
- Integrado dentro de la arquitectura SPA

9.5.9. Formulario de inicio de sesión



El sistema incluye un formulario de autenticación accesible desde la landing page mediante una ventana modal, permitiendo a los usuarios registrados acceder a las funcionalidades del sistema.

9.5.9.1. Campos del formulario

- **Usuario**
- **Contraseña**



9.5.9.2. Validaciones implementadas

- Verificación de credenciales contra el sistema
 - Control de errores:
 - ✗ Usuario o contraseña incorrectos → mensaje de error
 - ✓ Credenciales correctas → acceso al sistema
-

9.5.9.3. Funcionalidad

- Permite a los usuarios autenticarse en la aplicación
 - Habilita el acceso a las funcionalidades del control del robot
 - Gestiona la sesión del usuario
-

9.5.9.4. Características de diseño

- Implementado como **ventana modal** dentro de la landing page
- Interfaz sencilla y clara
- Feedback inmediato al usuario
- Integrado en la arquitectura SPA

9.5.10. Política de privacidad

El sistema incluye una sección de política de privacidad accesible desde la landing page, donde se informa al usuario sobre el tratamiento de sus datos personales dentro de la aplicación.

9.5.10.1. Contenido principal

- Información sobre el uso de datos personales:
 - Identificación del grupo o aula
 - Datos de contacto
 - Información técnica (IP, dispositivo, etc.)
- Finalidad del tratamiento:
 - Uso educativo
 - Mejora del servicio



- Protección de menores:
 - Tratamiento con consentimiento del responsable legal
 - Uso limitado a lo estrictamente necesario
-

9.5.10.2. Derechos del usuario

- Acceso a sus datos
- Rectificación
- Supresión

(Indicando contacto por correo electrónico)

9.5.10.3. Otros aspectos

- Uso de cookies limitado al funcionamiento de la sesión
 - Posibilidad de actualización de la política
 - Información de contacto para consultas
-

9.5.10.4. Funcionalidad

- Garantizar transparencia en el tratamiento de datos
- Cumplimiento de normativa básica de privacidad
- Generar confianza en el usuario

9.5.11. Contactanos

El sistema incluye una sección de contacto accesible desde la landing page, cuyo objetivo es proporcionar al usuario información para comunicarse con los responsables del proyecto.

9.5.11.1. Información mostrada

- **Proyecto:**
Dirección física asociada al desarrollo del sistema.



- **Teléfono:**

Número de contacto para consultas directas.

- **Correo electrónico:**

Dirección destinada a soporte, dudas o solicitudes relacionadas con el uso del sistema.

9.5.11.2. Funcionalidad

- Facilitar la comunicación entre usuarios y desarrolladores
 - Permitir resolver incidencias o dudas sobre el uso del robot
 - Ofrecer soporte técnico o educativo
-

9.5.11.3. Integración en la web

- Se presenta como una sección dentro de la landing page
- Mantiene el mismo diseño visual que el resto de la interfaz
- Está accesible desde el pie de página (footer)

9.6. API interna y comunicación

9.6.1. Mapa de Tópicos, Servicios y Mensajes

La interfaz web (patricio_web) se comunica con el robot a través de Rosbridge, un servidor WebSocket que escucha en el puerto 9090. La tabla siguiente distingue entre tópicos confirmados (verificados con `ros2 topic list`) y los pendientes de activar los módulos de juego y navegación.

9.6.2. Pantalla de caras del robot

Patricio dispone de una pantalla integrada que actúa como rostro digital del robot y constituye uno de los principales canales de comunicación emocional con el niño. La pantalla muestra en cada momento una expresión facial



coherente con el estado del robot y con la actividad que está realizando, reforzando así el vínculo afectivo con el usuario y haciendo que la interacción resulte más natural y atractiva para el público objetivo de cuatro a seis años.

9.6.2.1. Expresiones diseñadas

Se han diseñado un conjunto de expresiones faciales, cada una con su propio estilo gráfico coherente con la identidad visual del proyecto (tipografía DM Sans, paleta de colores definida en el moodboard y un fondo morado uniforme que aporta calidez a la pantalla). Las expresiones actualmente disponibles son las siguientes:

- Feliz (**happy**) — Cara sonriente con ojos cerrados arqueados y mejillas sonrosadas. Se muestra durante los juegos cuando el robot está disfrutando de la actividad o cuando ha completado correctamente una tarea. Se acompaña del texto *"Patricio está feliz"*.

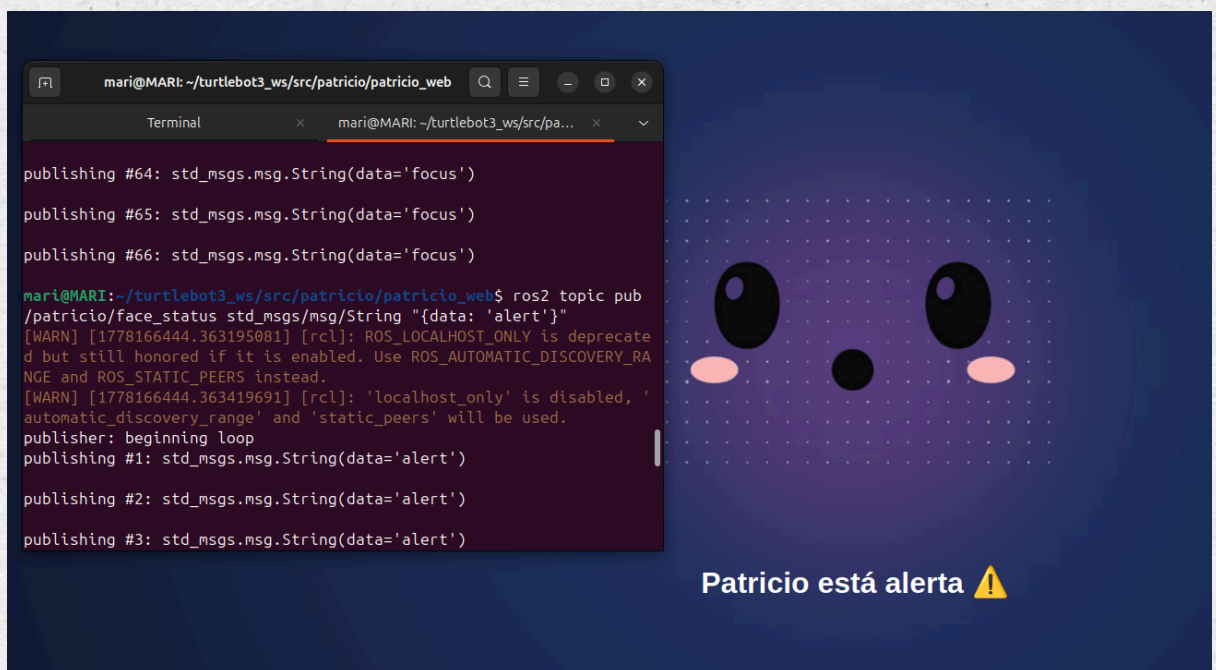


- Triste (**sad**) — Cara con cejas caídas, lágrimas en los ojos y boca temblorosa. Se muestra cuando el robot detecta una situación negativa (por ejemplo, al perder un juego o al detectar inactividad prolongada). Se acompaña del texto *"Patricio está triste"*.





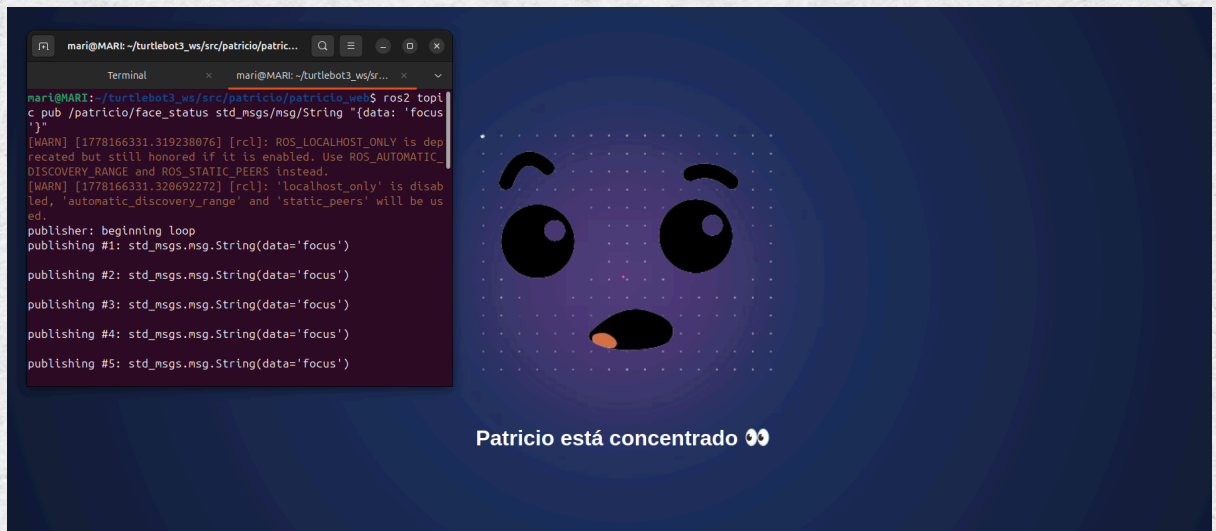
- Alerta (**alert**) — Cara con ojos muy abiertos y boca en forma de "o", indicando atención o sorpresa. Se muestra cuando el robot detecta un evento que requiere la intervención del usuario o cuando ocurre una incidencia. Se acompaña del texto "*Patricio está alerta*".



- Concentrado (**focus**) — Cara con ojos serios y mirada fija, indicando que el robot está procesando o ejecutando una tarea importante. Se muestra, por ejemplo, durante la planificación de una ruta de navegación o durante la búsqueda en el juego del escondite. Se acompaña del texto "*Patricio*"



está concentrado".



9.7. Indicador de actividad

Complementando a la expresión facial, en la parte inferior de la pantalla se muestra un indicador contextual que informa de la actividad concreta que está realizando el robot. Este indicador presenta un icono y un texto descriptivo, por ejemplo *"Jugando al Escondite"*, *"Jugando al Pilla-Pilla"* o *"Jugando al Calamar"*. La combinación de cara + indicador permite al adulto que supervisa la sesión saber de un vistazo tanto el estado emocional del robot como la actividad en curso.

9.8. Mecanismo de activación

La pantalla de caras se implementa como una página web ([face_screen.html](#)) que se ejecuta en el dispositivo de pantalla integrado en el robot. Esta página se suscribe a dos tópicos ROS 2 mediante `rosbridge`:

- `/patricio/face_status` (`std_msgs/String`) — Recibe el identificador de la expresión a mostrar (`happy`, `sad`, `alert`, `focus`).
- `/patricio/game_status` (`std_msgs/String`) — Recibe el identificador de la actividad en curso (`pilla_pilla`, `escondite`, `juego_calamar`).

De esta forma, cualquier nodo del sistema puede solicitar un cambio de expresión o un cambio de indicador publicando el mensaje correspondiente. Por ejemplo, el nodo `pilla_pilla_node` publica automáticamente `happy` en `/patricio/face_status` y `pilla_pilla` en `/patricio/game_status` cuando se inicia el juego, lo que provoca que la pantalla del robot se actualice de forma síncrona con la acción del sistema.

Este desacoplamiento entre la lógica de los juegos y la representación visual del



rostro permite añadir nuevas expresiones o nuevos juegos en el futuro sin necesidad de modificar los nodos existentes: basta con publicar el nuevo identificador en el tópico correspondiente y añadir el activo gráfico en [face_screen.html](#).

10. Tópicos

10.1. Tópicos — Confirmados en ejecución

Nombre del Tópico	Tipo de Mensaje ROS2	Dirección	Descripción
/cmd_vel	geometry_msgs/msg/Twist	Web → Robot	Control de velocidad lineal y angular. Canal principal de teleoperación.
/odom	nav_msgs/msg/Odometry	Robot → Web	Posición y orientación actuales del robot (odometría).
/scan	sensor_msgs/msg/LaserScan	Robot → Web	Lecturas del LiDAR. Visualización del entorno en la interfaz.
/imu	sensor_msgs/msg/Imu	Robot → Web	Datos de la unidad inercial (aceleración, giroscopio).



/joint_states	sensor_msgs/msg/JointState	Robot → Web	Estado de las articulaciones (posición, velocidad).
/tf	tf2_msgs/msg/TFMessage	Robot → Web	Árbol de transformadas en tiempo real.
/tf_static	tf2_msgs/msg/TFMessage	Robot → Web	Transformadas estáticas del modelo.
/robot_description	std_msgs/msg/String	Robot → Web	URDF del robot publicado por robot_state_publisher.
/clock	roscpp_msgs/msg/Clock	Simulador → Todos	Todos Reloj de simulación de Gazebo.
/client_count	std_msgs/msg/Int32	Rosbridge → Web	Número de clientes WebSocket conectados.
/connected_clients	rosbridge_msgs/msg/ConnectedClients	Rosbridge → Web	Detalle de los clientes WebSocket conectados.

10.2. Tópicos — Pendientes (módulos de juego/navegación)

Estos tópicos aparecerán al ejecutar: `ros2 run patricio_pilla_pilla`

`pilla_pilla_node`



Nombre del Tópico	Tipo de Mensaje ROS2	Dirección	Descripción
/patricio/pilla_pilla/cmd	geometry_msgs/msg/PoseStamped	Web → Robot	Posición del jugador durante el juego de pilla-pilla.
/patricio/pilla_pilla/status	std_msgs/msg/String	Robot → Web	Estado: idle, running, robot_wins,

			player_wins.
/patricio/nav_punto/goal	geometry_msgs/msg/PoseStamped	Web → Robot	Destino para navegación autónoma punto a punto.
/patricio/nav_ruta waypoints	nav_msgs/msg/Path	Web → Robot	Lista de poses que definen una ruta completa.



/patricio/face/expressi on	std_msgs/msg/String	Web → Robot	Expresi ón facial: happy, sad, surpris ed, neutral.
/patricio/speech/say	std_msgs/msg/String	Web → Robot	Texto para síntesis de voz (TTS).

11. Nodos activos confirmados

Nodo	Función
/robot_state_publisher	Publica la descripción URDF y el árbol TF del robot.
/ros_gz_bridge	Puente entre Gazebo y ROS 2. Traduce mensajes del simulador.
/rosbridge_websocket	Servidor WebSocket que conecta la web con ROS 2.
/rosapi	Servicio auxiliar de Rosbridge para introspección de tópicos.

12. Servicios (Rosbridge / Rosapi)

Nombre del Servicio	Tipo ROS 2	Descripción
/rosapi/topics	rosapi_msgs/srv/Topics	Lista de tópicos activos. Usado



		internamente por la web.
/rosapi/nodes	rosapi_msgs/srv/Nodes	Lista de nodos activos.
/rosapi/message_details	rosapi_msgs/srv/MessageDetails	Definición de un tipo de mensaje concreto.
/patricio/pilla_pilla/start	std_srvs/srv/Trigger	Inicia el juego de pilla-pilla. (pendiente de módulo)
/patricio/pilla_pilla/stop	std_srvs/srv/Trigger	Detiene el juego. (pendiente de módulo)
/patricio/nav_punto/cancel	std_srvs/srv/Trigger	Cancela la navegación en curso. (pendiente de módulo)

13. Ejemplos de JSON desde JavaScript

Todos los mensajes se envían al WebSocket de Rosbridge como objetos JSON con el campo "op" indicando la operación (publish, subscribe, call_service).

Mensaje PoseStamped — /patricio/pilla_pilla/cmd

Enviado desde JS para indicar la posición del jugador durante el juego de



pilla-pilla:

```
{  
  "op": "publish",  
  "topic": "/patricio/pilla_pilla/cmd",  
  "msg": {  
    "header": {  
      "stamp": { "sec": 1745000000, "nanosec": 0 },  
      "frame_id": "map"  
    },  
    "pose": {  
      "position": { "x": 2.5, "y": 1.3, "z": 0.0 },  
      "orientation": { "x": 0.0, "y": 0.0, "z": 0.7071, "w": 0.7071 }  
    }  
  }  
}
```

La orientación se expresa como cuaternión. z: 0.7071, w: 0.7071 = 90° sobre el eje Z. Para orientación neutra (0°) usar z: 0.0, w: 1.0.

Mensaje Twist — /cmd_vel (confirmado activo)

Teleoperación directa del robot desde la interfaz web:

```
{
```



```
"op": "publish",  
"topic": "/cmd_vel",  
"msg": {  
  "linear": { "x": 0.5, "y": 0.0, "z": 0.0 },  
  "angular": { "x": 0.0, "y": 0.0, "z": 0.3 }  
}  
}
```

Snippet JavaScript con roslibjs

```
const ros = new ROSLIB.Ros({ url: 'ws://<IP_ROBOT>:9090' });  
// Teleoperación (ACTIVO)
```

```
const cmdVel = new ROSLIB.Topic({  
  ros, name: '/cmd_vel', messageType: 'geometry_msgs/Twist'  
});
```

```
cmdVel.publish(new ROSLIB.Message({  
  linear: { x: 0.5, y: 0.0, z: 0.0 },  
  angular: { x: 0.0, y: 0.0, z: 0.3 }  
}));
```

```
// Navegación a un punto (requiere módulo nav_punto)
```

```
const navGoal = new ROSLIB.Topic({  
  ros, name: '/patricio/nav_punto/goal',  
  messageType: 'geometry_msgs/PoseStamped'  
});
```

```
navGoal.publish(new ROSLIB.Message({
```



```
header: { stamp: { sec: Math.floor(Date.now()/1000), nanosec: 0 }, frame_id:
'map' }, pose: {
position: { x: 2.5, y: 1.3, z: 0.0 },
orientation: { x: 0.0, y: 0.0, z: 0.7071, w: 0.7071 }
}
}});
```

14. Lista de Dependencias del Sistema

A continuación se listan todos los componentes necesarios para replicar el entorno del proyecto Patricio desde cero.

14.1.Sistema Operativo y ROS 2

Dependencia	Versión	Notas
Ubuntu	24.04LTS (Noble)	Requerido para ROS 2 Jazzy
Ros2 Jazzy Jalisco	jazzy	Distribución LTS 2024- 2029
Python	3.12+	Incluido en Ubuntu 24.04
CMake	3.8+	Para compliar paquetes C++



14.2. Paquetes ROS 2 Core

```
sudo apt install ros-jazzy-desktop  
sudo apt install ros-jazzy-std-msgs  
sudo apt install ros-jazzy-geometry-msgs  
sudo apt install ros-jazzy-sensor-msgs  
sudo apt install ros-jazzy-nav-msgs  
sudo apt install ros-jazzy-std-srvs  
sudo apt install ros-jazzy-tf2-ros  
sudo apt install ros-jazzy-robot-state-publisher  
sudo apt install ros-jazzy-joint-state-publisher
```

14.3. Rosbridge — Confirmado activo

```
sudo apt install ros-jazzy-rosbridge-suite  
# Lanzar WebSocket en puerto 9090:  
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

14.4. Simulador Gazebo — Confirmado activo

```
sudo apt install ros-jazzy-ros-gz  
sudo apt install ros-jazzy-ros-gz-bridge  
sudo apt install ros-jazzy-ros-gz-sim
```



14.5. Navegación Nav2

```
sudo apt install ros-jazzy-navigation2
```

```
sudo apt install ros-jazzy-nav2-bringup
```

```
sudo apt install ros-jazzy-nav2-msgs
```

14.6. Librerías WEB - Frontend

Librería JS	Descripción
roslibjs	Cliente JS para comunicarse con Rosbridge vía WebSocket
ros2djs	Visualización 2D de mapas y poses del robot
eventemitter2	Dependencia interna de roslibjs

```
<script
```

```
src="https://cdn.jsdelivr.net/npm/roslib/build/roslib.min.js"></script>
```

```
<script
```

```
src="https://cdn.jsdelivr.net/npm/ros2d/build/ros2d.min.js"></script>
```

15. Instalación completa desde cero

15.1. Activar ROS 2

```
source /opt/ros/jazzy/setup.bash
```



15.2. Clonar y compilar

```
git clone https://github.com/CJMIDNIGHT/patricio.git
```

```
~/ros2_ws/src/patricio cd ~/ros2_ws
```

```
rosdep update
```

```
rosdep install --from-paths src --ignore-src -r -y
```

```
colcon build --symlink-install
```

```
source install/setup.bash
```

15.3. Lanzar simulación (Terminal 1)

```
ros2 launch patricio_my_world house.launch.py
```

15.4. Lanzar Rosbridge (Terminal 2)

```
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

15.5. Acceder a la interfaz web

```
# http://<IP_ROBOT>:8080
```

16. Verificación del sistema (pruebas y validación)

16.1. Lista de pruebas

Prueba	Descripción	Resultado
H7 Juego Pilla pilla - P1	Inicio y ejecución del juego	Correcto. El juego se inicia correctamente y el robot comienza a navegar entre los puntos establecidos



H7 Juego Pilla pilla - P2	Finalización del juego	Correcto. El juego se detiene correctamente tras enviar la orden, cancelando la navegación
H10 Juego Escondite - P1	Inicio y ejecución del juego	Correcto. El juego se inicia correctamente y el robot comienza la búsqueda
H10 Juego Escondite - P2	Finalización del juego	Correcto. El juego se detiene correctamente al recibir la orden
H11 Página WEB - P1	Conexión a robot desde la web exitosa	Correcto. La conexión se establece correctamente y el estado cambia a "Conectado"
H11 Página WEB - P2	Login correcto	Correcto. El usuario accede correctamente al sistema mostrando mensaje de éxito
H11 Página WEB - P3	Login incorrecto	Correcto. Se muestra mensaje de error indicando credenciales incorrectas
H11 Página WEB - P4	Cerrar sesión	Correcto. Se solicita confirmación y se cierra la sesión correctamente
H11 Página WEB - P5	Comunicación entre la web y la API	Correcto. Las peticiones se realizan correctamente y la API responde con estado 200
H14 Navegación autónoma	Desplazamiento a un punto	Correcto. El robot navega hasta el punto indicado y permite cancelar la navegación



16.1.1. H11-P1 Conexión a robot desde la web

Descripción:

Se verifica que la interfaz web requiere conexión previa con el robot antes de permitir cualquier acción.

Procedimiento:

1. Abrir la interfaz web
2. Intentar iniciar un juego o mover el robot sin pulsar el botón de conexión
3. Observar el comportamiento
4. Pulsar el botón "Conectar"
5. Repetir la acción

Resultado esperado:

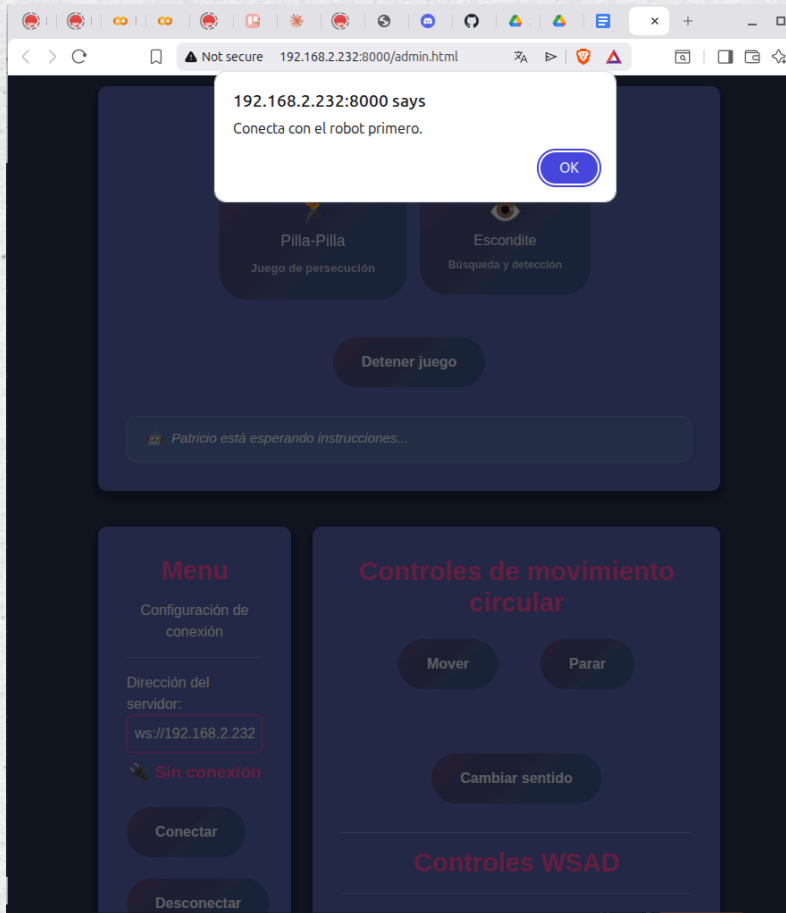
- Sin conexión: se muestra un mensaje indicando que el usuario debe conectarse al robot
- Con conexión: las acciones funcionan correctamente

Resultado obtenido:

Correcto

Evidencia:





16.1.2. H7-P1 - Iniciar el juego Pilla-pilla

Descripción:

Se verifica que el sistema inicia correctamente el juego Pilla-Pilla desde la interfaz web y que el robot comienza a navegar entre los waypoints generados.

Procedimiento:

1. Conectar con el robot desde la web
2. Pulsar el botón “Pilla-Pilla”
3. Observar el estado del robot en la interfaz y en RViz

Resultado esperado:

- El juego se inicia correctamente
- El robot comienza a moverse entre diferentes puntos
- Se muestra el estado “Corriendo” en la interfaz

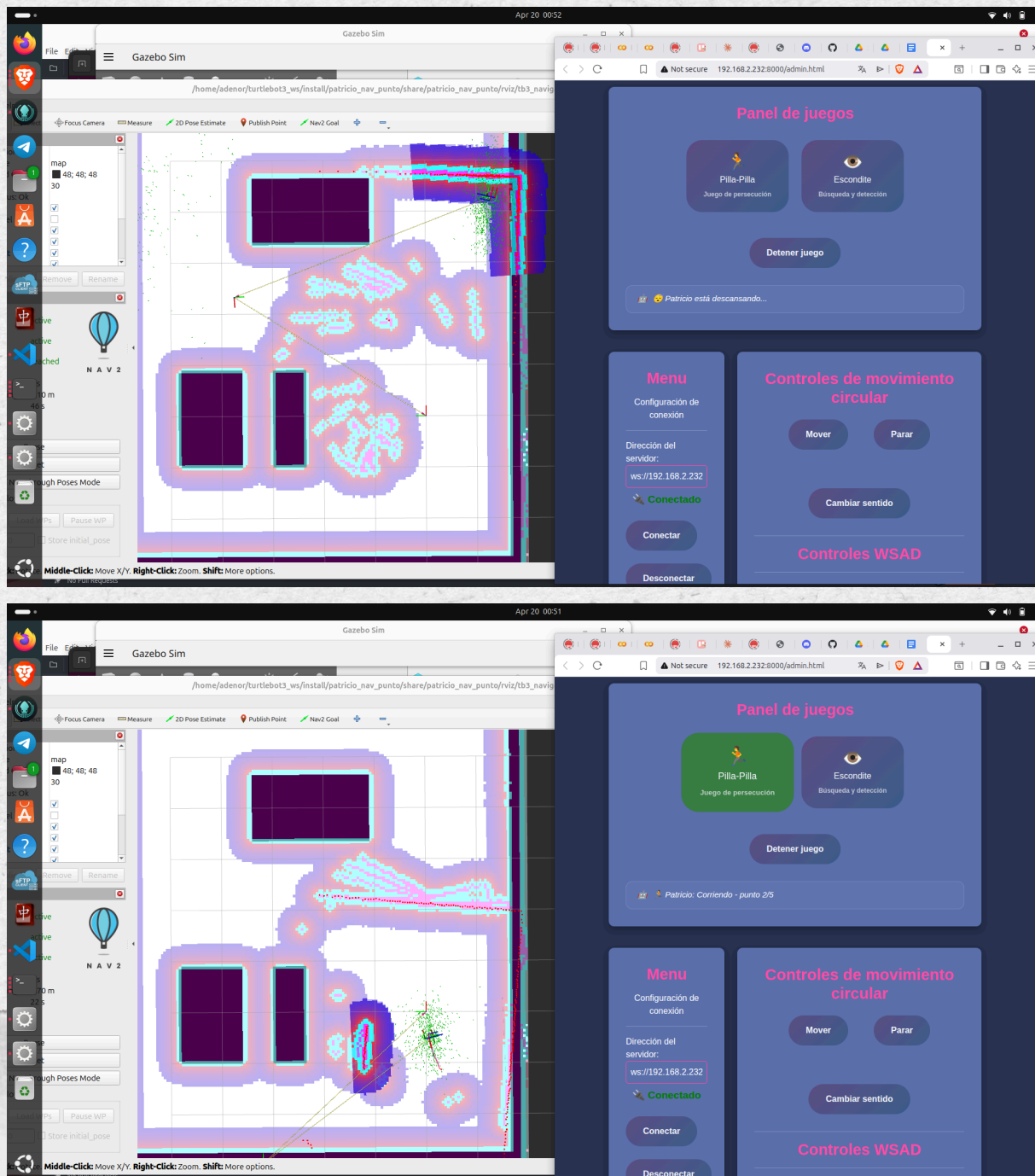


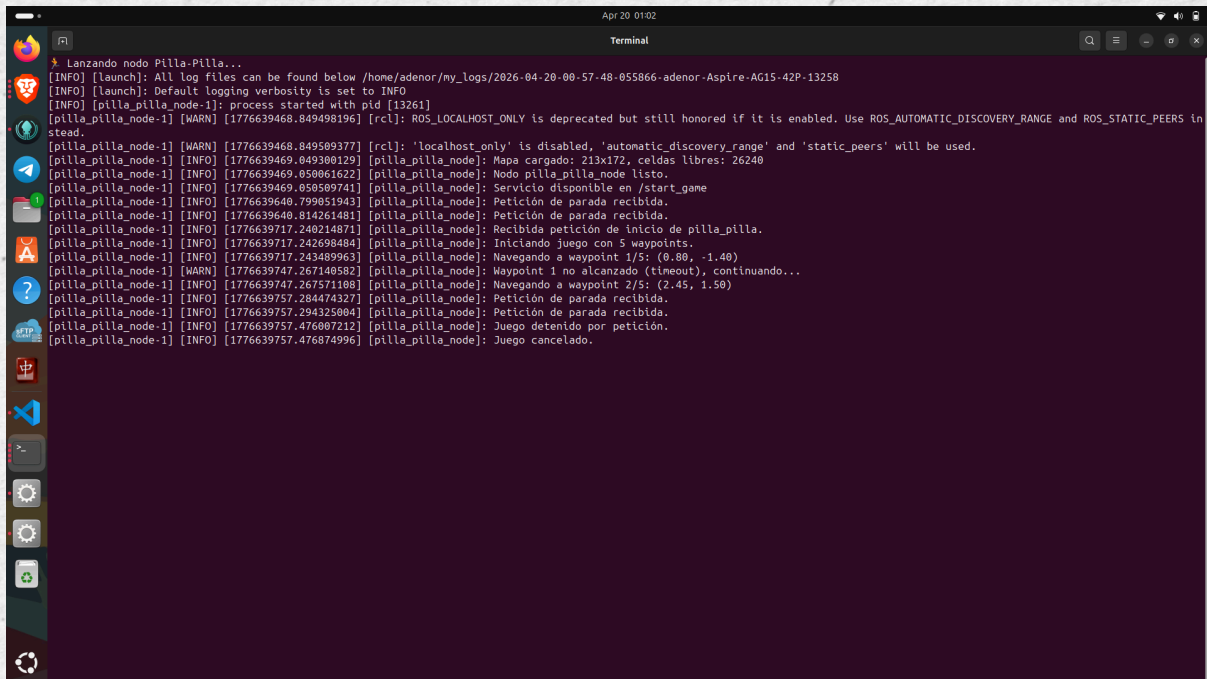
Resultado obtenido:

Correcto. El robot inicia la navegación entre los waypoints y la interfaz muestra el estado del juego en ejecución.

Evidencia:

- Interfaz mostrando estado “Corriendo”
- Movimiento del robot en RViz
- Logs indicando inicio del juego





```
Apr 20 01:02
Terminal
Lanzando nodo Pilla-Pilla...
[INFO] [launch]: All log files can be found below /home/adenor/my_logs/2026-04-20-00-57-48-055866-adenor-Aspire-AG15-42P-13258
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [pilla_pilla_node-1]: process started with pid [13261]
[pilla_pilla_node-1] [WARN] [1776639468.849498196] [rcl]: ROS_LOCALHOST_ONLY is deprecated but still honored if it is enabled. Use ROS_AUTOMATIC_DISCOVERY_RANGE and ROS_STATIC_PEERS instead.
[pilla_pilla_node-1] [WARN] [1776639468.849509377] [rcl]: 'localhost_only' is disabled, 'automatic_discovery_range' and 'static_peers' will be used.
[pilla_pilla_node-1] [INFO] [1776639469.049308129] [pilla_pilla_node]: Mapa cargado: 213x172, celdas libres: 26240
[pilla_pilla_node-1] [INFO] [1776639469.050061622] [pilla_pilla_node]: Nodo pilla_pilla_node listo.
[pilla_pilla_node-1] [INFO] [1776639469.050509741] [pilla_pilla_node]: Servicio disponible en /start_game
[pilla_pilla_node-1] [INFO] [1776639640.799051943] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639640.814261481] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639717.240214871] [pilla_pilla_node]: Recibida petición de inicio de pilla_pilla.
[pilla_pilla_node-1] [INFO] [1776639717.242698404] [pilla_pilla_node]: Iniciando juego con 5 waypoints.
[pilla_pilla_node-1] [INFO] [1776639717.243489963] [pilla_pilla_node]: Navegando a waypoint 1/5: (0.80, -1.40)
[pilla_pilla_node-1] [WARN] [1776639747.267140582] [pilla_pilla_node]: Waypoint 1 no alcanzado (timeout), continuando...
[pilla_pilla_node-1] [INFO] [1776639747.267571108] [pilla_pilla_node]: Navegando a waypoint 2/5: (2.45, 1.50)
[pilla_pilla_node-1] [INFO] [1776639757.284474327] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639757.294325004] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639757.476007212] [pilla_pilla_node]: Juego detenido por petición.
[pilla_pilla_node-1] [INFO] [1776639757.476874996] [pilla_pilla_node]: Juego cancelado.
```

16.1.3. H7-P2 - Detener el juego Pilla-pilla

Descripción:

Se verifica que el sistema puede detener el juego en ejecución mediante la interfaz web.

Procedimiento:

1. Iniciar el juego Pilla-Pilla
2. Pulsar el botón “Detener juego”

Resultado esperado:

- El robot detiene la navegación
- El estado cambia a “Descansando”
- Se cancela la tarea actual en Nav2

Resultado obtenido:

Correcto. El sistema cancela la navegación en curso y el robot deja de moverse.

Evidencia:

- Logs indicando “Juego detenido”
- Cancelación de tarea en navegación



- Estado actualizado en la web

```

Lanzando nodo Pilla.Pilla...
[INFO] [launch]: All log files can be found below /home/adenor/my_logs/2026-04-20-00-57-48-055866-adenor-Aspire-AG15-42P-13258
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [pilla_pilla_node-1]: process started with pid [13261]
[pilla_pilla_node-1] [WARN] [1776639468.849498196] [rccl]: ROS_LOCALHOST_ONLY is deprecated but still honored if it is enabled. Use ROS_AUTOMATIC_DISCOVERY_RANGE and ROS_STATIC_PEERS instead.
[pilla_pilla_node-1] [WARN] [1776639468.849509377] [rccl]: 'localhost_only' is disabled, 'automatic_discovery_range' and 'static_peers' will be used.
[pilla_pilla_node-1] [INFO] [1776639469.049380129] [pilla_pilla_node]: Mapa cargado: 213x172, celdas libres: 26248
[pilla_pilla_node-1] [INFO] [1776639469.050061622] [pilla_pilla_node]: Nodo pilla_pilla node listo.
[pilla_pilla_node-1] [INFO] [1776639469.050509741] [pilla_pilla_node]: Servicio disponible en /start_game
[pilla_pilla_node-1] [INFO] [1776639640.799051943] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639640.814261481] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639717.240214871] [pilla_pilla_node]: Recibida petición de inicio de pilla_pilla.
[pilla_pilla_node-1] [INFO] [1776639717.242698484] [pilla_pilla_node]: Iniciando juego con 5 waypoints.
[pilla_pilla_node-1] [INFO] [1776639717.243489963] [pilla_pilla_node]: Navegando a waypoint 1/5: (0.80, -1.40)
[pilla_pilla_node-1] [WARN] [1776639747.267140502] [pilla_pilla_node]: Waypoint 1 no alcanzado (timeout), continuando...
[pilla_pilla_node-1] [INFO] [1776639747.267571188] [pilla_pilla_node]: Navegando a waypoint 2/5: (2.45, 1.50)
[pilla_pilla_node-1] [INFO] [1776639757.284474327] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639757.294325084] [pilla_pilla_node]: Petición de parada recibida.
[pilla_pilla_node-1] [INFO] [1776639757.476007212] [pilla_pilla_node]: Juego detenido por petición.
[pilla_pilla_node-1] [INFO] [1776639757.476874996] [pilla_pilla_node]: Juego cancelado.
  
```

16.1.4. H11-P1 - Iniciar el juego Escondite

Descripción:

Se verifica que el sistema inicia correctamente el juego del escondite desde la interfaz web y que el robot comienza la búsqueda del objetivo.

Procedimiento:

1. Conectar con el robot desde la web
2. Seleccionar varios puntos en el mapa (mínimo 2)
3. Pulsar el botón “Escondite”

Resultado esperado:

- El juego se inicia correctamente
- El robot navega entre los puntos definidos
- Se muestra el estado “Buscando” en la interfaz

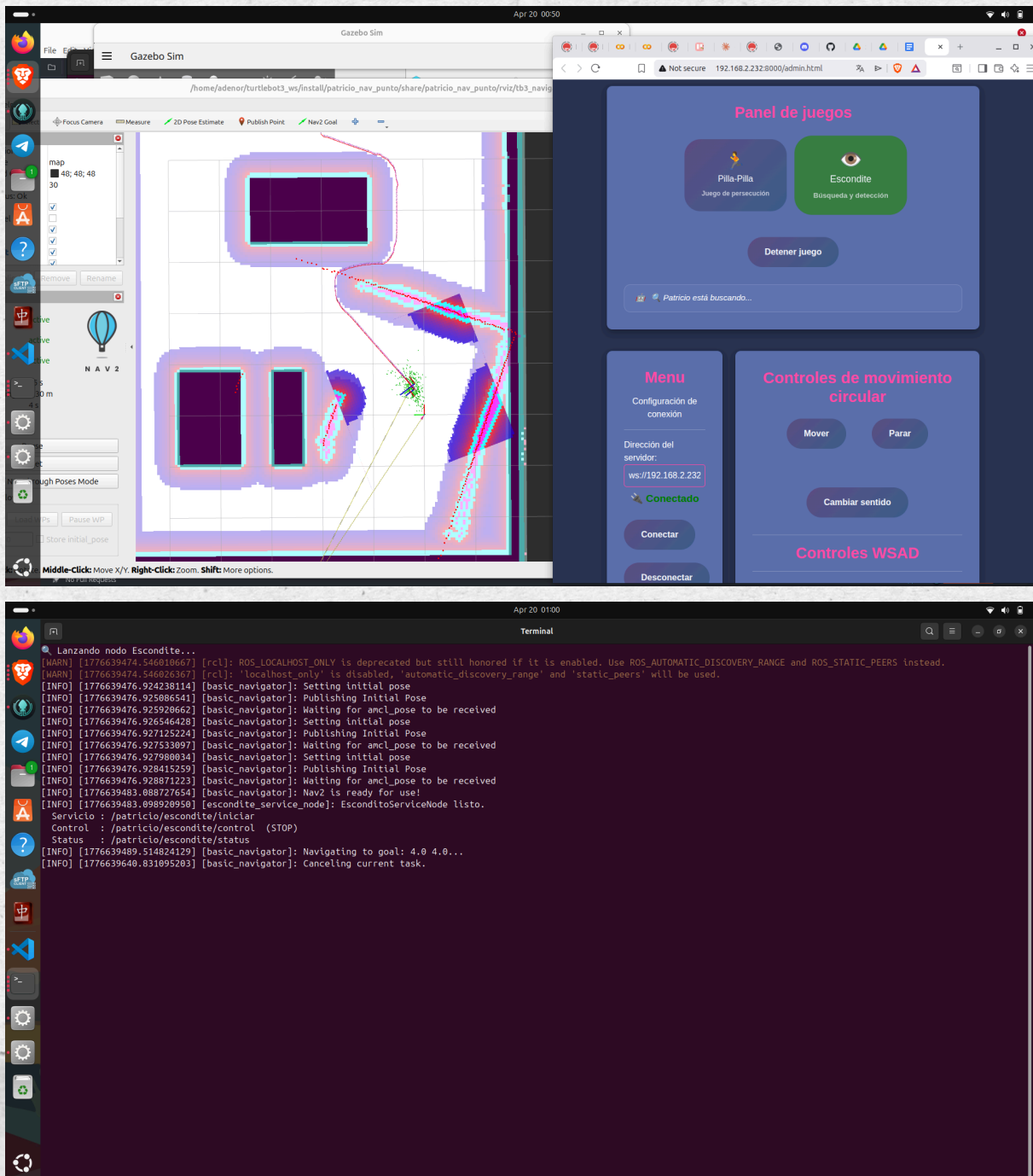
Resultado obtenido:

Correcto. El robot inicia la navegación recorriendo los puntos y la interfaz muestra el estado del juego en ejecución.



Evidencia:

- Interfaz mostrando “Patricio está buscando...”
- Movimiento del robot en RViz
- Logs indicando navegación activa



16.1.5. H11-P2 - Detener el juego Escondite

Descripción:

Se verifica que el sistema puede cancelar la búsqueda en cualquier momento.

Procedimiento:

1. Iniciar el juego Escondite
2. Pulsar el botón "Detener juego"

Resultado esperado:

- El robot detiene la navegación
- Se cancela la tarea actual en Nav2
- El sistema vuelve a estado de reposo

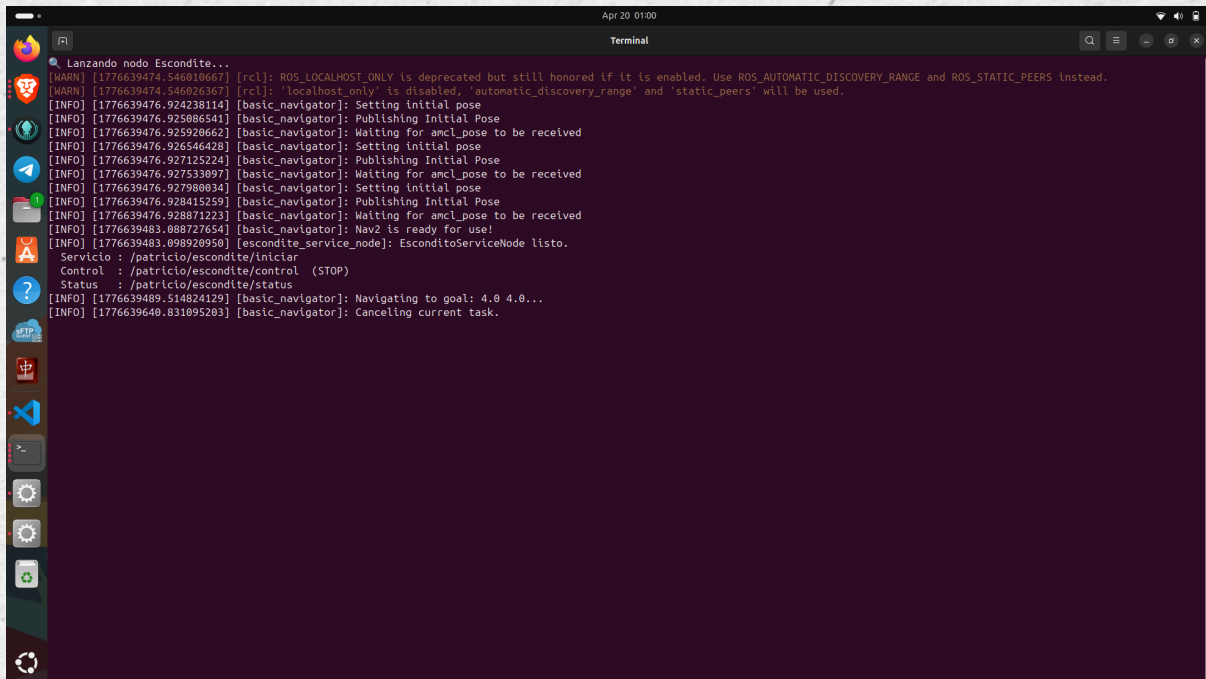
Resultado obtenido:

Correcto. El sistema cancela la navegación en curso mediante el servicio y el robot deja de moverse.

Evidencia:

- Logs mostrando "Canceling current task"
- Detención del robot en RViz





```
Lanzando nodo Escondite...
[WARN] [1776639474.546810667] [rccl]: ROS_LOCALHOST_ONLY is deprecated but still honored if it is enabled. Use ROS_AUTOMATIC_DISCOVERY_RANGE and ROS_STATIC_PEERS instead.
[WARN] [1776639474.546826367] [rccl]: 'localhost_only' is disabled, 'automatic_discovery_range' and 'static_peers' will be used.
[INFO] [1776639476.924238114] [basic_navigator]: Setting initial pose
[INFO] [1776639476.925886541] [basic_navigator]: Publishing Initial Pose
[INFO] [1776639476.925928662] [basic_navigator]: Waiting for ancl_pose to be received
[INFO] [1776639476.926346428] [basic_navigator]: Setting initial pose
[INFO] [1776639476.927125224] [basic_navigator]: Publishing Initial Pose
[INFO] [1776639476.927533897] [basic_navigator]: Waiting for ancl_pose to be received
[INFO] [1776639476.927988034] [basic_navigator]: Setting initial pose
[INFO] [1776639476.928415259] [basic_navigator]: Publishing Initial Pose
[INFO] [1776639476.928871223] [basic_navigator]: Waiting for ancl_pose to be received
[INFO] [1776639483.088727654] [basic_navigator]: Nav2 is ready for use!
[INFO] [1776639483.088920958] [escondite_service_node]: EsconditoServiceNode listo.
Servicio : /patricio/escondite/iniciar
Control : /patricio/escondite/control (STOP)
Status : /patricio/escondite/status
[INFO] [1776639489.514824129] [basic_navigator]: Navigating to goal: 4.0 4.0...
[INFO] [1776639648.831895203] [basic_navigator]: Canceling current task.
```

16.1.6. H11-P2 - Login correcto

Descripción:

Se verifica el comportamiento del sistema cuando el usuario introduce credenciales válidas en la aplicación web.

Procedimiento:

1. Acceder a la pantalla de inicio de sesión
2. Introducir un usuario y contraseña correctos
3. Pulsar el botón “Entrar”

Resultado esperado:

- El sistema permite el acceso
- Se muestra un mensaje de confirmación
- El usuario accede al panel principal de la aplicación

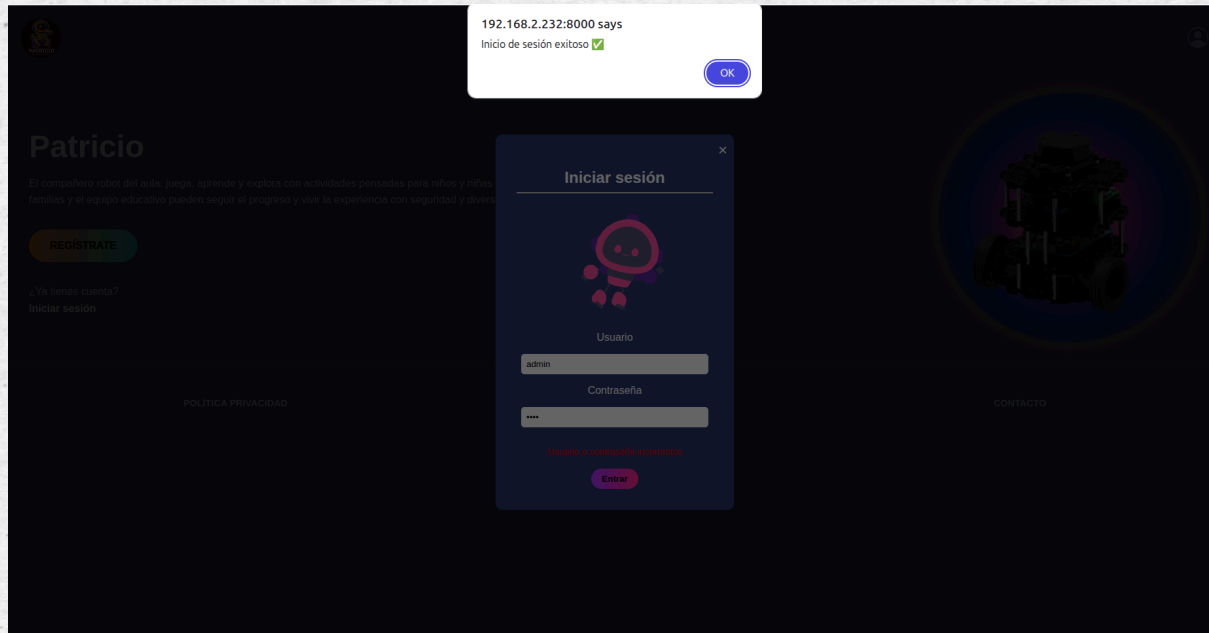
Resultado obtenido:

Correcto. El sistema muestra el mensaje “Inicio de sesión exitoso” y permite el acceso al sistema.



Evidencia:

La siguiente imagen muestra el mensaje de confirmación tras introducir credenciales válidas:



16.1.7. H11-P3 - Login incorrecto

Descripción:

Se verifica el comportamiento del sistema cuando el usuario introduce credenciales incorrectas al intentar iniciar sesión en la aplicación web.

Procedimiento:

1. Acceder a la pantalla de inicio de sesión
2. Introducir un usuario o contraseña incorrectos
3. Pulsar el botón “Entrar”

Resultado esperado:

- El sistema no permite el acceso
- Se muestra un mensaje de error indicando credenciales incorrectas
- El usuario permanece en la pantalla de login

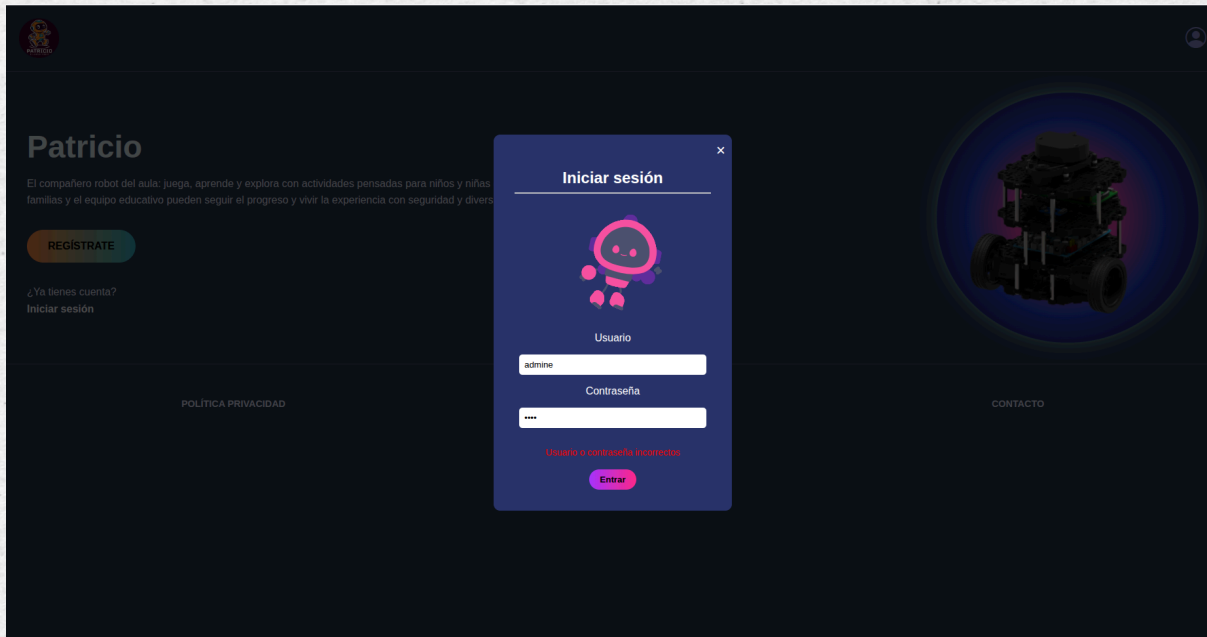


Resultado obtenido:

Correcto. El sistema muestra el mensaje "Usuario o contraseña incorrectos" y no permite el acceso.

Evidencia:

La siguiente imagen muestra el mensaje de error tras introducir credenciales inválidas:



16.1.8. H11-P4 - Cerrar sesión

Descripción:

Se verifica que el sistema permite cerrar sesión correctamente desde la interfaz web, incluyendo una confirmación previa para evitar acciones accidentales.

Procedimiento:

1. Acceder a la interfaz web
2. Pulsar el botón "Cerrar sesión"
3. Observar la aparición del mensaje de confirmación
4. Seleccionar la opción "Sí" o "No"

Resultado esperado:

- Al pulsar "Cerrar sesión", aparece un mensaje de confirmación
- Si el usuario selecciona "Sí", se cierra la sesión



- Si selecciona “No”, permanece en la aplicación

Resultado obtenido:

Correcto. El sistema muestra una ventana de confirmación antes de cerrar sesión, evitando cierres accidentales.

Evidencia:

La siguiente imagen muestra el mensaje de confirmación al intentar cerrar sesión:

16.1.9. H11-P5 - Comunicación entre la web y la API**Descripción:**

Se verifica que la aplicación web se comunica correctamente con la API backend mediante peticiones HTTP para iniciar, detener y consultar el estado de los juegos.

Procedimiento:

1. Iniciar la API (patricio_api.py)
 2. Acceder a la interfaz web
 3. Ejecutar acciones desde la web:
 - Iniciar juego (Pilla-Pilla o Escondite)
 - Consultar estado del juego
 - Detener juego
-

Resultado esperado:

- La API recibe las peticiones correctamente
 - Se devuelven respuestas HTTP con código 200
 - Se actualiza el estado del juego en la web
-

Resultado obtenido:

Correcto. La API recibe múltiples peticiones (POST y GET) desde la web,

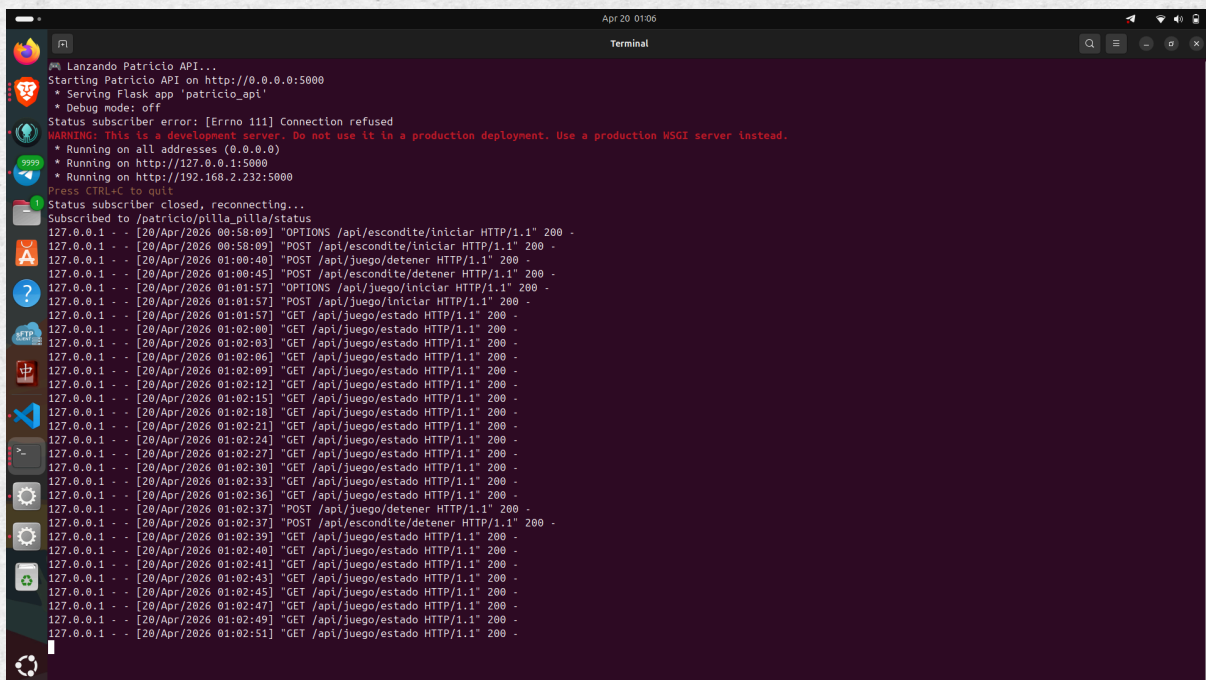


respondiendo con código 200 en todos los casos y gestionando correctamente el estado del juego.

Evidencia:

En la siguiente imagen se observan las peticiones realizadas a la API:

- POST /api/juego/iniciar → inicio del juego
- GET /api/juego/estado → consulta del estado
- POST /api/juego/detener → parada del juego
- POST /api/escondite/iniciar → inicio escondite



```
Apr 20 01:06
Terminal
Lanzando Patricio API...
Starting Patricio API on http://0.0.0.0:5000
* Serving Flask app 'patricio_api'
* Debug mode: off
Status subscriber error: [Errno 111] Connection refused
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.2.232:5000
Press CTRL-C to quit
Status subscriber closed, reconnecting...
Subscribed to /patricio/pilla pilla/status
127.0.0.1 - - [20/Apr/2026 00:58:09] "OPTIONS /api/escondite/iniciar HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 00:58:09] "POST /api/escondite/iniciar HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:00:40] "POST /api/juego/detener HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:00:45] "POST /api/escondite/detener HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:01:57] "OPTIONS /api/juego/iniciar HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:01:57] "POST /api/juego/iniciar HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:01:57] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:00] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:03] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:06] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:09] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:12] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:15] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:18] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:21] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:24] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:27] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:30] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:33] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:36] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:37] "POST /api/juego/detener HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:37] "POST /api/escondite/detener HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:39] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:40] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:41] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:43] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:45] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:47] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:49] "GET /api/juego/estado HTTP/1.1" 200 -
127.0.0.1 - - [20/Apr/2026 01:02:51] "GET /api/juego/estado HTTP/1.1" 200 -
```

16.1.10. H14-P1 - Navegar a un punto

Descripción:

Se verifica que el robot es capaz de desplazarse de forma autónoma hasta un punto objetivo utilizando el sistema de navegación Nav2.

Procedimiento:

1. Lanzar el entorno de simulación en Gazebo



2. Iniciar el sistema de navegación (Nav2)
3. Establecer la pose inicial del robot en RViz
4. Enviar un objetivo de navegación
5. Observar el comportamiento del robot

Resultado esperado:

El robot planifica una ruta y se desplaza hasta el punto indicado evitando obstáculos.

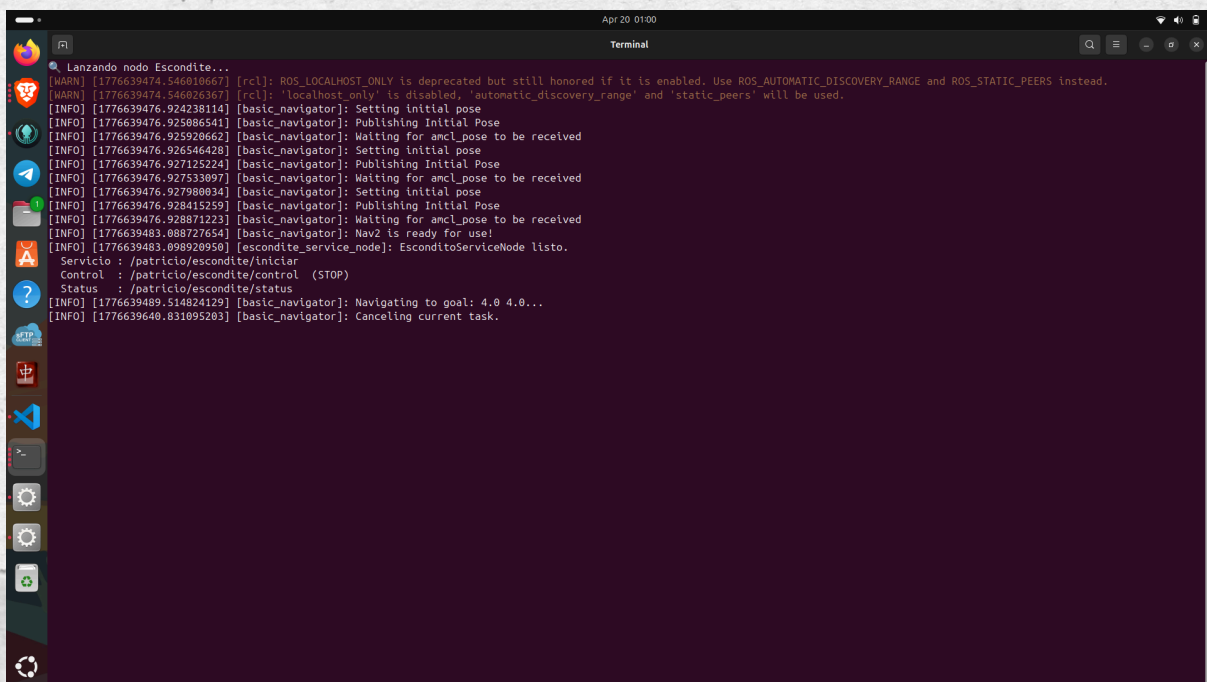
Resultado obtenido:

Correcto. El robot inicia la navegación hacia el objetivo y el sistema Nav2 responde adecuadamente.

Evidencia:

La siguiente captura muestra la ejecución del sistema, donde se observa el inicio de la navegación:

- Inicialización correcta de Nav2
- Publicación de la pose inicial
- Inicio de navegación hacia el objetivo



```
Apr 20 01:00
Terminal
Lanzando nodo Escondite...
[WARN] [1776639474.546818667] [rcl]: ROS_LOCALHOST_ONLY is deprecated but still honored if it is enabled. Use ROS_AUTOMATIC_DISCOVERY_RANGE and ROS_STATIC_PEERS instead.
[WARN] [1776639474.546826367] [rcl]: 'localhost_only' is disabled, 'automatic_discovery_range' and 'static_peers' will be used.
[INFO] [1776639476.924238114] [basic_navigator]: Setting initial pose
[INFO] [1776639476.925886541] [basic_navigator]: Publishing Initial Pose
[INFO] [1776639476.925920662] [basic_navigator]: Waiting for amcl_pose to be received
[INFO] [1776639476.926546428] [basic_navigator]: Setting initial pose
[INFO] [1776639476.927125224] [basic_navigator]: Publishing Initial Pose
[INFO] [1776639476.927533897] [basic_navigator]: Waiting for amcl_pose to be received
[INFO] [1776639476.927988834] [basic_navigator]: Setting initial pose
[INFO] [1776639476.928415259] [basic_navigator]: Publishing Initial Pose
[INFO] [1776639476.928871223] [basic_navigator]: Waiting for amcl_pose to be received
[INFO] [1776639483.088727654] [basic_navigator]: Nav2 is ready for use!
[INFO] [1776639483.098920958] [escondite_service_node]: EsconditeServiceNode listo.
Servicio : /patricio/escondite/iniciar
Control : /patricio/escondite/control (STOP)
Status : /patricio/escondite/status
[INFO] [1776639489.514824129] [basic_navigator]: Navigating to goal: 4.0 4.0...
[INFO] [1776639640.831095203] [basic_navigator]: Canceling current task.
```



17. Troubleshooting

A continuación se describen los problemas más comunes detectados durante el uso del sistema y sus posibles soluciones.

17.1. La web muestra "Disconnected"

- Verificar que rosbridge está en ejecución:
 - `ros2 launch rosbridge_server rosbridge_websocket_launch.xml`
- Comprobar que el puerto 9090 está activo
- Revisar la IP introducida en la web (ws://<IP>:9090)
- Comprobar firewall o red

17.2. No conecta la web con el robot

- Asegurarse de haber lanzado rosbridge
- Verificar que el robot/simulación está en ejecución
- Revisar conexión de red entre cliente y robot

17.3. El robot no se mueve

- Verificar que la navegación está lanzada:
 - `ros2 launch patricio_nav_punto my_navigation.launch.py`
- Comprobar que se ha hecho "2D Pose Estimate" en RViz
- Verificar que el mapa está cargado correctamente
- Revisar que `/cmd_vel` recibe mensajes



17.4. El juego Pilla-Pilla no inicia

- Verificar que el nodo está activo:
 - `ros2 run patricio_pilla_pilla pilla_pilla_node`
- Comprobar que el servicio `/start_game` está disponible
- Revisar logs en consola

17.5. El robot no llega a los waypoints

- **Comprobar tolerancia de llegada**
- **Verificar que los puntos están en zona navegable del mapa**
- **Revisar si hay obstáculos en el entorno**

17.6. No se ve el mapa en la web

- Verificar que `/map` está siendo publicado
- Comprobar conexión con `rosbridge`
- Revisar errores en consola del navegador

17.7. El login no funciona

- **Verificar credenciales introducidas**
- **Revisar conexión con la API Flask**
- **Comprobar que el backend está activo**

18. Conclusión

Patricio es mucho más que un robot; es el resultado de un enfoque integral que combina tecnología, empatía y sostenibilidad para transformar la experiencia educativa de miles de niños , especialmente de aquell@s en situación de déficit de atención o dificultad de atención.



A lo largo del desarrollo del proyecto, hemos identificado una necesidad real en el sector y hemos dado respuesta a través de una solución innovadora, escalable y adaptable, que mejora la eficiencia del aprendizaje y la calidad de vida de los usuarios. Nuestra propuesta se basa en una arquitectura técnica sólida con ROS 2, simulación en Gazebo, planificación autónoma con Nav2, y una interfaz web que permite la interacción desde múltiples canales.

Además, Patricio incorpora valores fundamentales como la accesibilidad, la transparencia y el impacto social positivo, apostando por un modelo decrecimiento responsable, escalable y alineado con los Objetivos de Desarrollo Sostenible.

Con este proyecto no solo demostramos el poder de la robótica aplicada a la educación, sino también la capacidad de un equipo comprometido para idear soluciones que realmente importan. Patricio no pretende reemplazar al ser humano, sino apoyarlo, liberar su tiempo y contribuir a una atención más justa, eficaz y digna.

Conclusiones personales y reflexiones del sprint:

A continuación se recogen las valoraciones individuales de cada miembro del equipo sobre el desarrollo del proyecto y el funcionamiento del grupo de trabajo durante este sprint.

Adenor Buret

Hasta ahora el sprint está yendo bien (no se rompió nada por la gracia de dios). Lo que más valoro es que cuando alguien se ha atascado con algo técnico lo ha dicho enseguida y entre todos hemos buscado la solución sin perder demasiado tiempo. Creo que por ahora vamos por el buen camino para cerrar el proyecto.

Santiago Aguirre Crespo

De momento el sprint está siendo bastante fluido. El reparto de tareas al inicio fue claro y eso ha facilitado que cada uno trabaje sin pisar al otro. Ha



habido algún momento de integración técnica que nos ha costado más de lo esperado, pero siempre se ha resuelto en equipo sin que se enquistara. El ambiente de trabajo es bueno y eso se nota en el ritmo.

Pablo Meana

Mi valoración del sprint hasta ahora es positiva. Las tareas están bien repartidas y el equipo está respondiendo bien cuando hay que coordinar entre módulos. Lo que más me ha gustado es que nadie ha esperado a que le dijeran qué hacer, cada uno ha tomado la iniciativa en su área. Todavía queda trabajo por delante pero veo al equipo con capacidad para terminarlo bien.

Mari Dapcheva

El grupo está funcionando bien en este sprint. Se trabaja de forma ordenada y cuando hay dudas o bloqueos se comunican rápido, lo cual evita que las tareas se acumulen. Lo que más valoro es que el ambiente es colaborativo de verdad. Cuando alguien pide ayuda el resto responde sin problema y eso hace que el trabajo sea mucho más llevadero.

César Herrero

Hasta aquí el sprint va bien. El equipo ha demostrado capacidad para trabajar de forma autónoma y para tomar decisiones técnicas con criterio cuando ha hecho falta. Las reuniones de seguimiento han sido útiles para detectar bloqueos a tiempo y redistribuir carga si era necesario. Estoy satisfecho con cómo estamos avanzando.

Juan Bautista Peris

Mi experiencia en este sprint está siendo buena. El ritmo es constante y no se está acumulando trabajo para el final. Cada vez que ha surgido un problema, técnico o de coordinación, se ha abordado enseguida. Creo que el equipo está madurando bien y que las tareas que quedan pendientes se van a resolver con la misma dinámica que llevamos hasta ahora.



